

Semi-Automatic Information and Knowledge Systems

L2: Advanced Ontology Modeling

SS 2026

Fajar J. Ekaputra

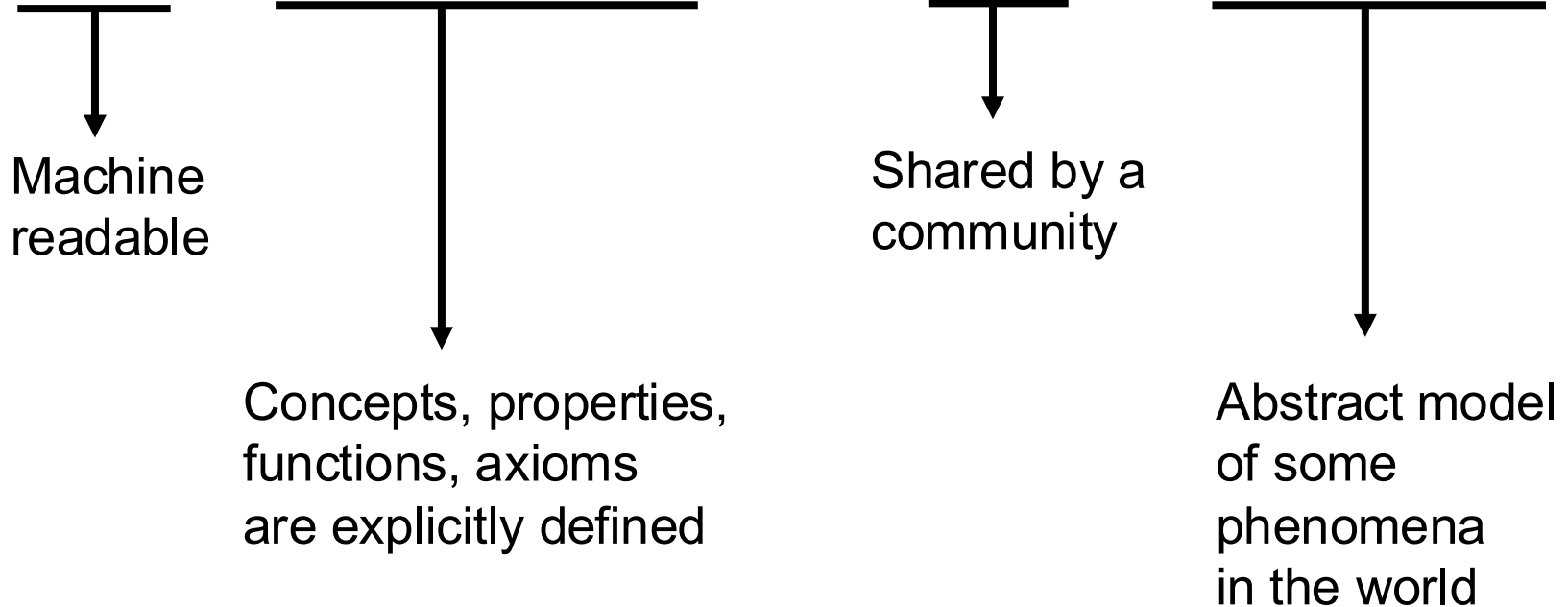
Day	Date ↕	Time	Location ↕	Approval	Description ↕
Fri	08.03.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Introduction to Advanced Semantic Applications (Ekaputra)
Fri	13.03.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Ontology Modeling (Advanced OWL, Ontology Design Patterns) (Ekaputra)
Fri	20.03.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Evaluation of Knowledge Structures (Ekaputra)
Fri	27.03.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	LLM for Knowledge Engineering (Ekaputra)
Fri	17.04.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Mid-term presentation
Fri	24.04.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Introduction to Ontology Alignment (Lanzenberger)
Mon	27.04.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Ontology Alignment in practice (Lanzenberger)
Mon	18.05.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Ontology Alignment in practice II (Lanzenberger)
Fri	22.05.2026	10:00 - 12:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	Semantic Integration (Lanzenberger)
Fri	12.06.2026	10:00 - 13:00	Seminarraum FAV 01 C (Seminarraum 188/2)	✓	End-Term Presentations of Practical Exercise

- **Ontologies – refresher**
- Advanced OWL Features
 - OWL2 – selected new features
 - Property restrictions
 - Property chaining
- Ontology Design Patterns
 - Modeling “part of”
 - Modeling n-ary relations
 - Modeling roles

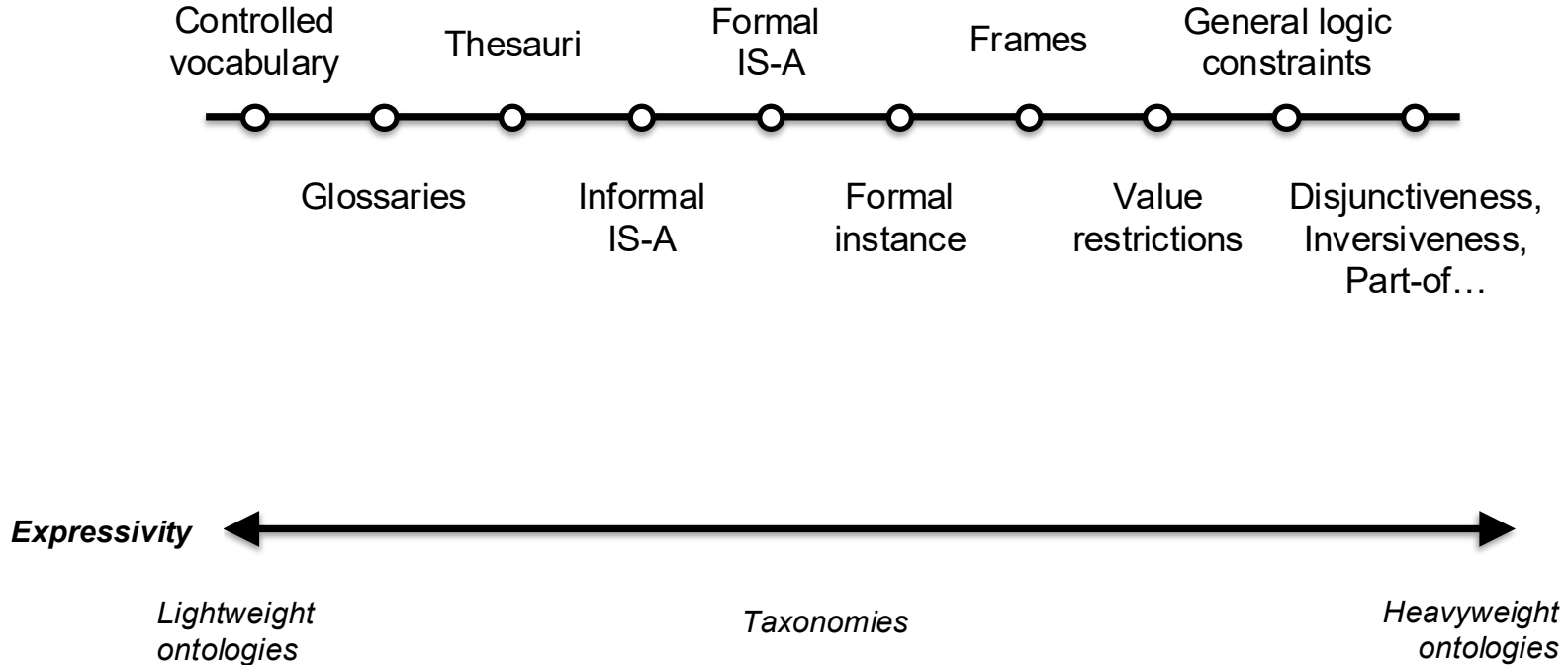
What is an ontology?

Studer(98):

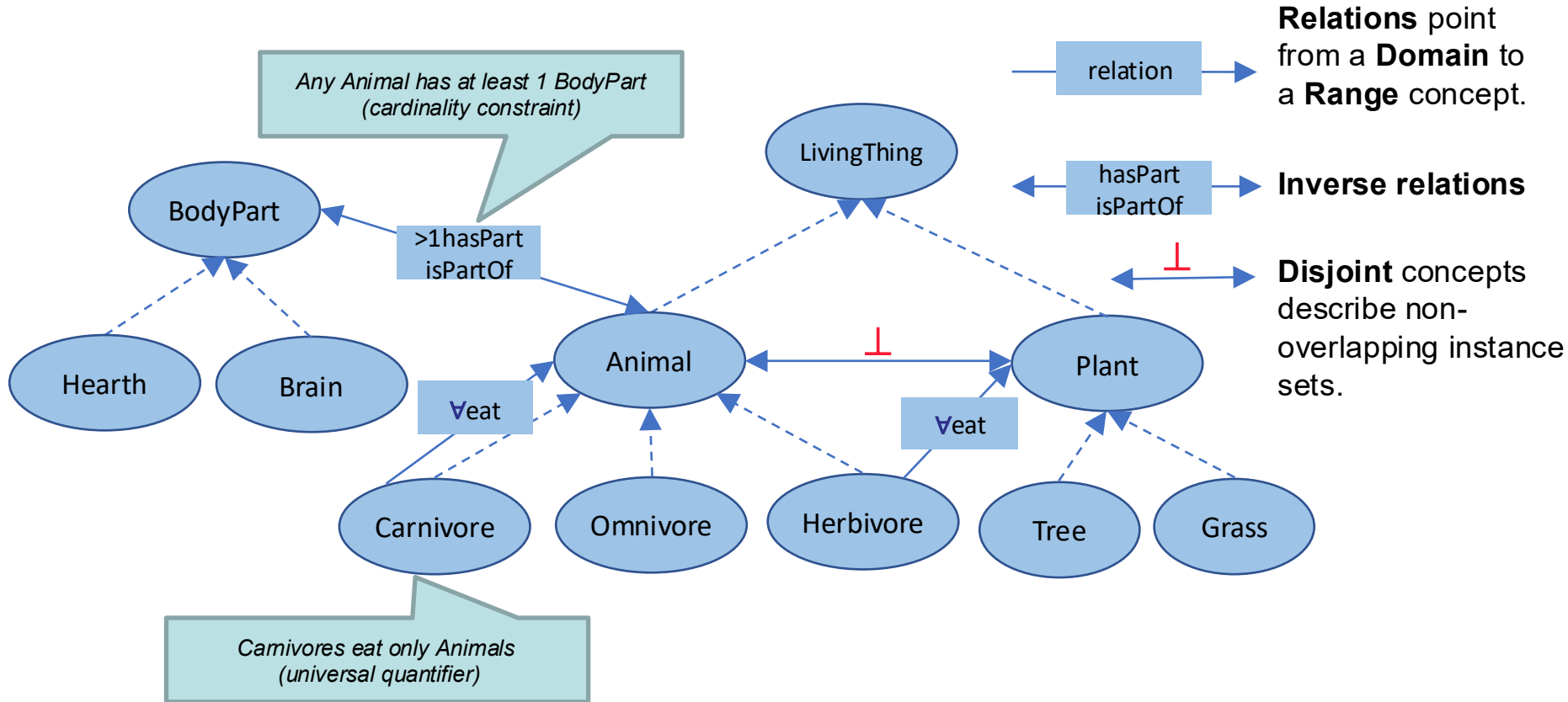
Formal, explicit specification of a shared conceptualization.



Ontology Types and Categories: Overview



Ontology: A taxonomy extended with other relations and further constraints.

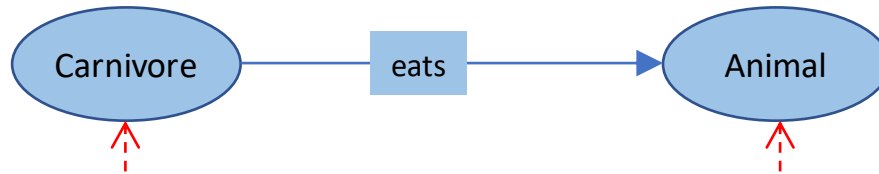




Metadata to describe the image:

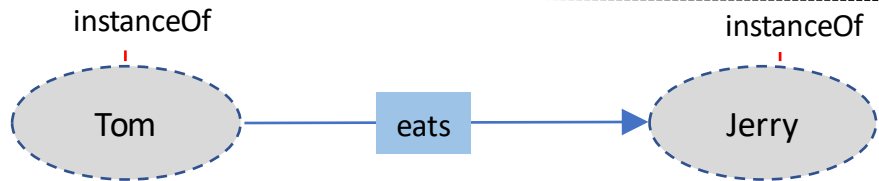
- *Tom is a Carnivore*
- *Jerry is an Animal*
- *Tom eats Jerry*

Terminological
Space



Instance represents an entity in the universe of discourse.

Instance
Space



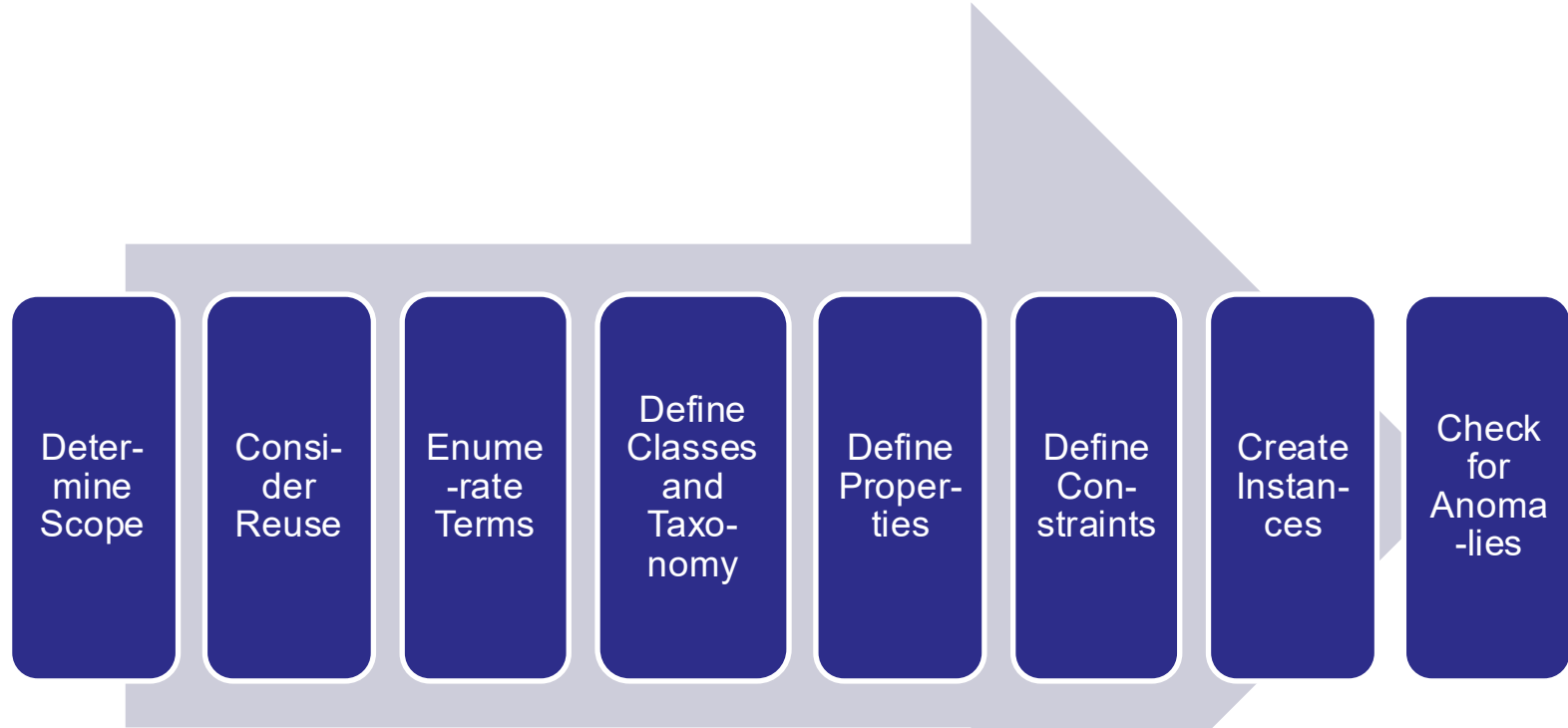
instanceOf → Types the instance with respect to a concept (the instance is of type concept).

Recap: What's inside an ontology?

- Concepts:
 - Denote the main concepts of the domain
 - *E.g.: Carnivore, Animal, Pump, Engine*
- Concept hierarchy
 - Denotes specialization/generalizations
 - *E.g.: Carnivore is a kind of Animal*
- Relations between classes
 - *E.g.: Carnivore eats Animal*
- Restrictions on relations (type, cardinality)
 - *E.g.: any Animal has at least one BodyPart*
- Instances:
 - Denote concrete entities in the domain
 - *E.g.: Tom, Jerry*

Ontology Development 101

(Noy and McGuinness, 2000)

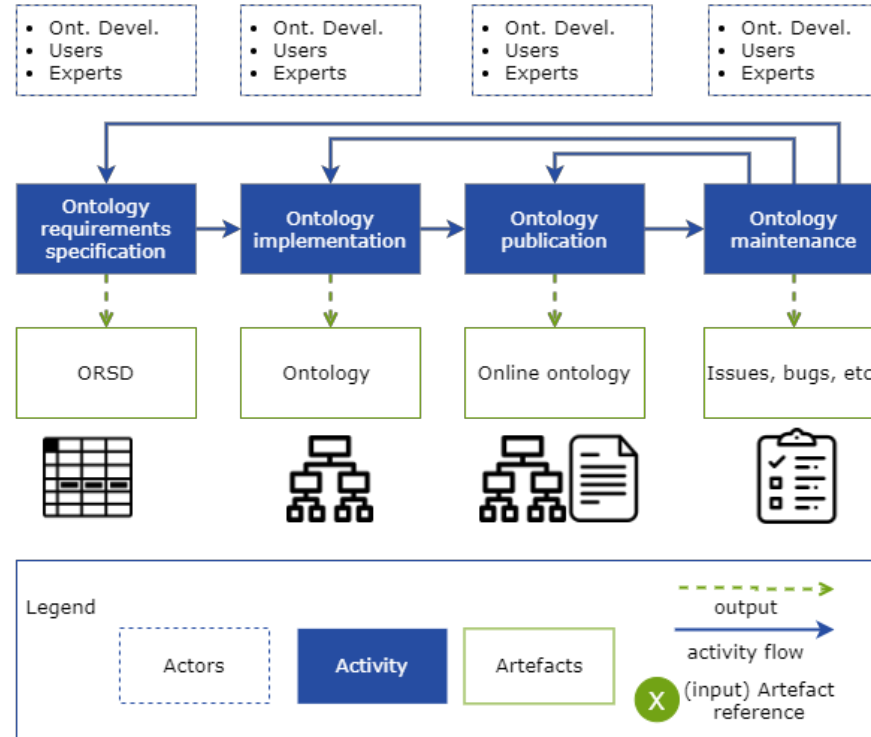


Not a linear process!

N. Noy, and D. McGuinness. *Ontology Development 101: A Guide to Creating Your First Ontology*, March 2001

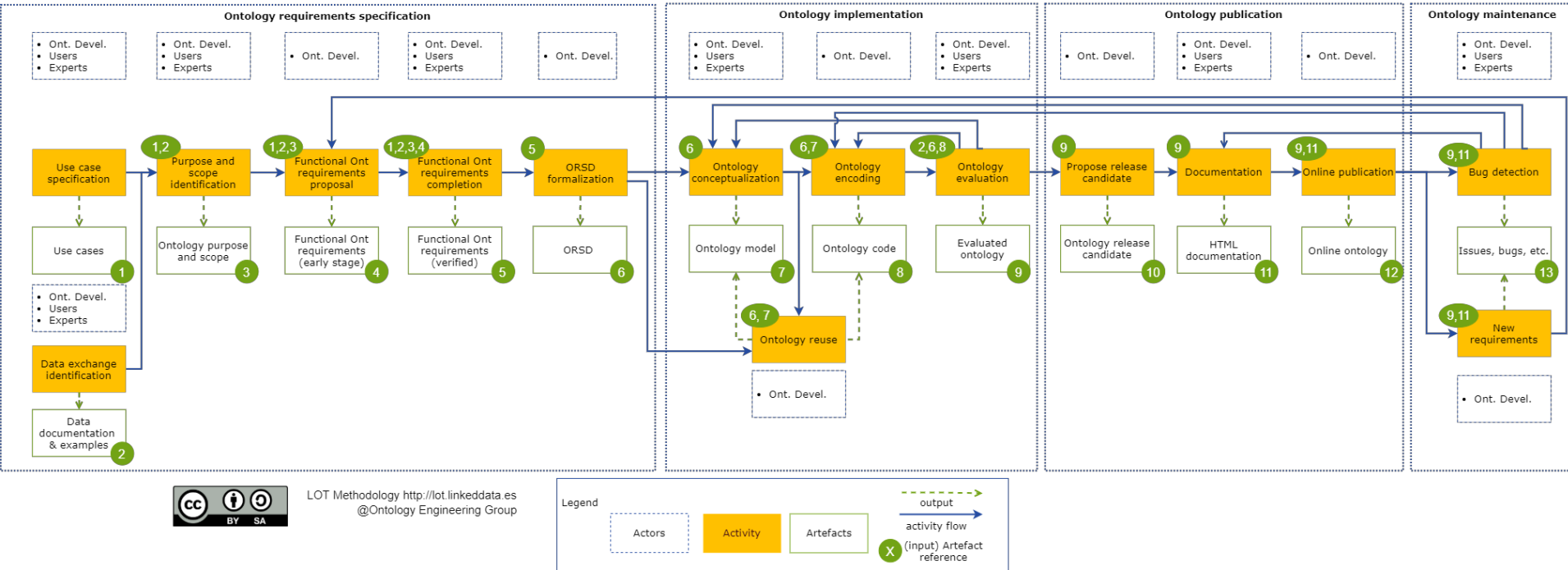
ReadingMaterial

Linked Open Terms (LOT) - an industrial oriented ontology engineering framework



LOT Methodology <http://lot.linkeddata.es>
@Ontology Engineering Group

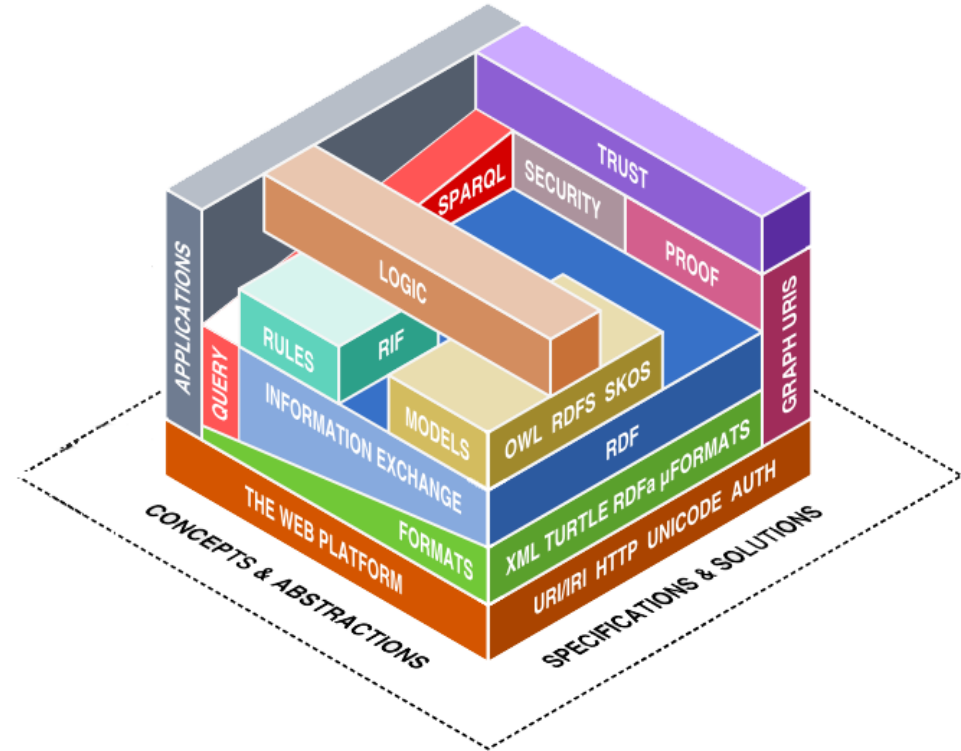
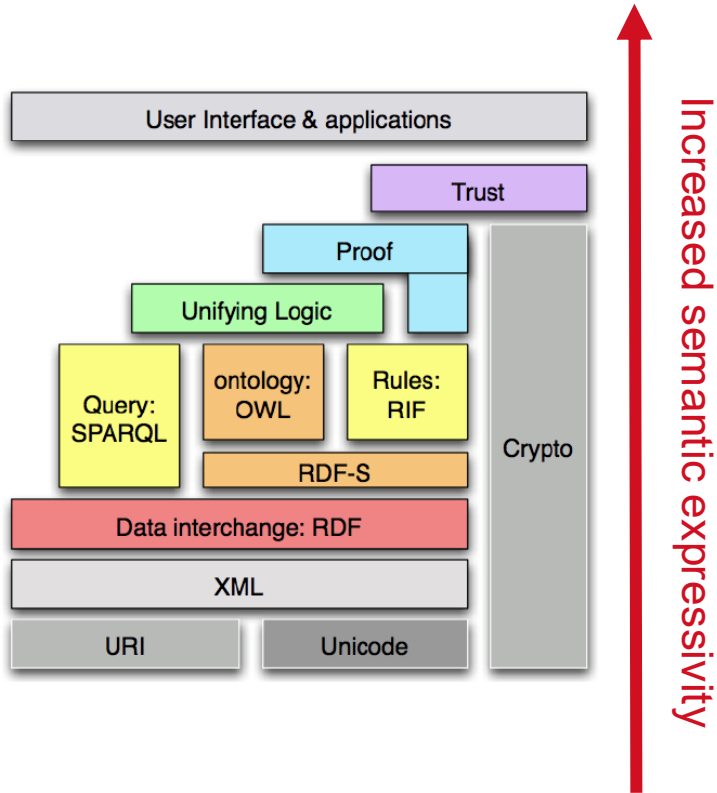
Linked Open Terms (LOT) - an industrial oriented ontology engineering framework



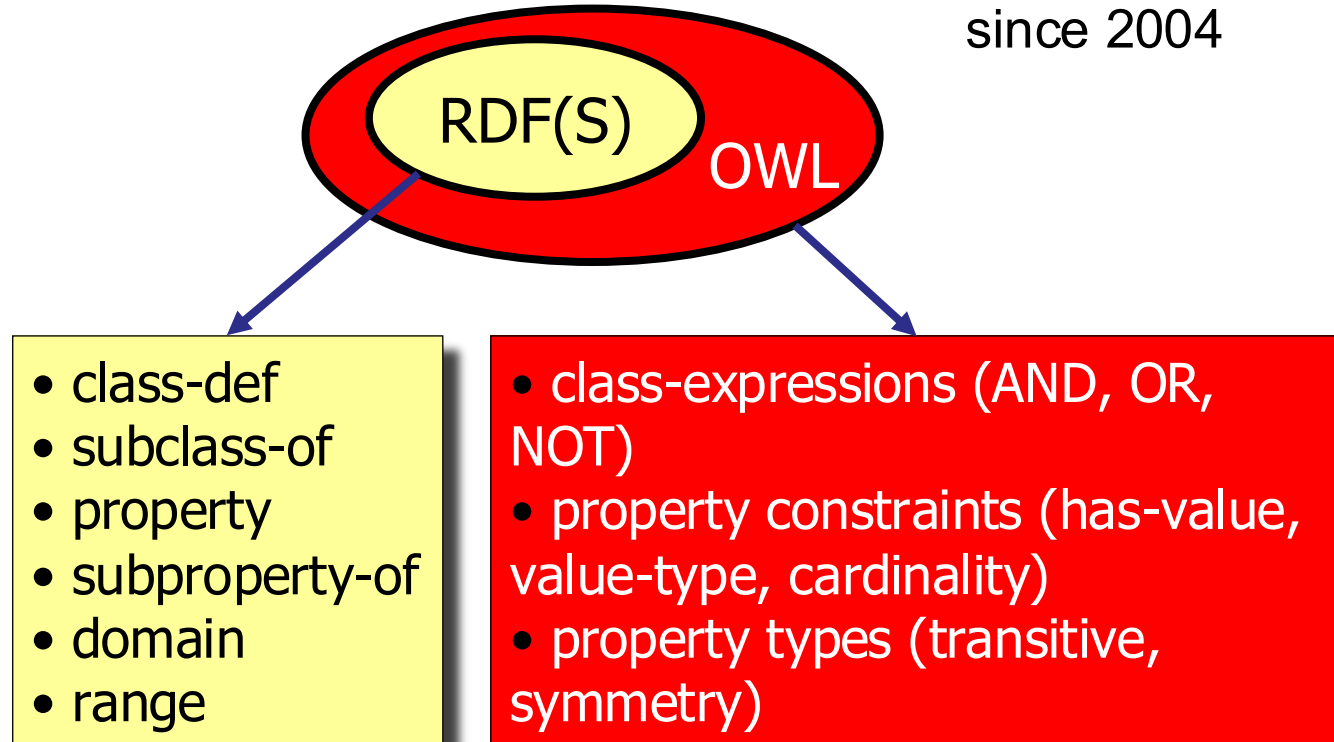
LOT Methodology <http://lot.linkeddata.es>
@Ontology Engineering Group

- Ontologies – refresher
- Advanced OWL Features
 - **OWL2 – selected new features**
 - Property restrictions
 - Property chaining
- Ontology Design Patterns
 - Modeling “part of”
 - Modeling n-ary relations
 - Modeling roles

Standard Syntax, Knowledge Representation Languages



W3C Recommendation
since 2004



© Mike Ushold

- W3C Recommendation since 2009
- Extensions based on practical experiences with OWL 1
 - Additional expressivity based on new ontological axioms
 - Additional minor extensions
 - New syntax variants, comments, etc.
- As far as possible compatible with OWL1

“OWL 2: The Next Step for OWL - Web Semantics”, Bernardo Cuenca Grau, Ian Horrocks, Boris Motik, Bijan Parsia, Peter Patel-Schneider, Ulrike Sattler.

ReadingMaterial

OWL 2 allows the creation of individuals as named entities, also without direct class membership

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:description rdf:about="MartaSabou">
    <rdf:type rdf:resource="&owl:NamedIndividual"/>
  </rdf:description>
</rdf:RDF>
```

- in turtle syntax...

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@base <http://example.org/>
```

```
<MartaSabou> a owl:NamedIndividual .
```

- OWL 1 and 2:
 - Two classes can be declared as disjoint classes (owl:disjointWith)
- OWL 2 allows an abbreviatory mechanism for declaring a set of classes as disjoint classes

```
<owl:AllDisjointClasses>
  <owl:members rdf:parseType="Collection">
    <owl:Class rdf:about="Plant" />
    <owl:Class rdf:about="Animal" />
    <owl:Class rdf:about="Woman" />
    <owl:Class rdf:about="Man" />
  </owl:members>
</owl:AllDisjointClasses>
```

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@base <http://example.org/> .

[] a owl:AllDisjointClasses ;
  owl:members (
    <Plant>
    <Animal>
    <Woman>
    <Man>
  ) .
```

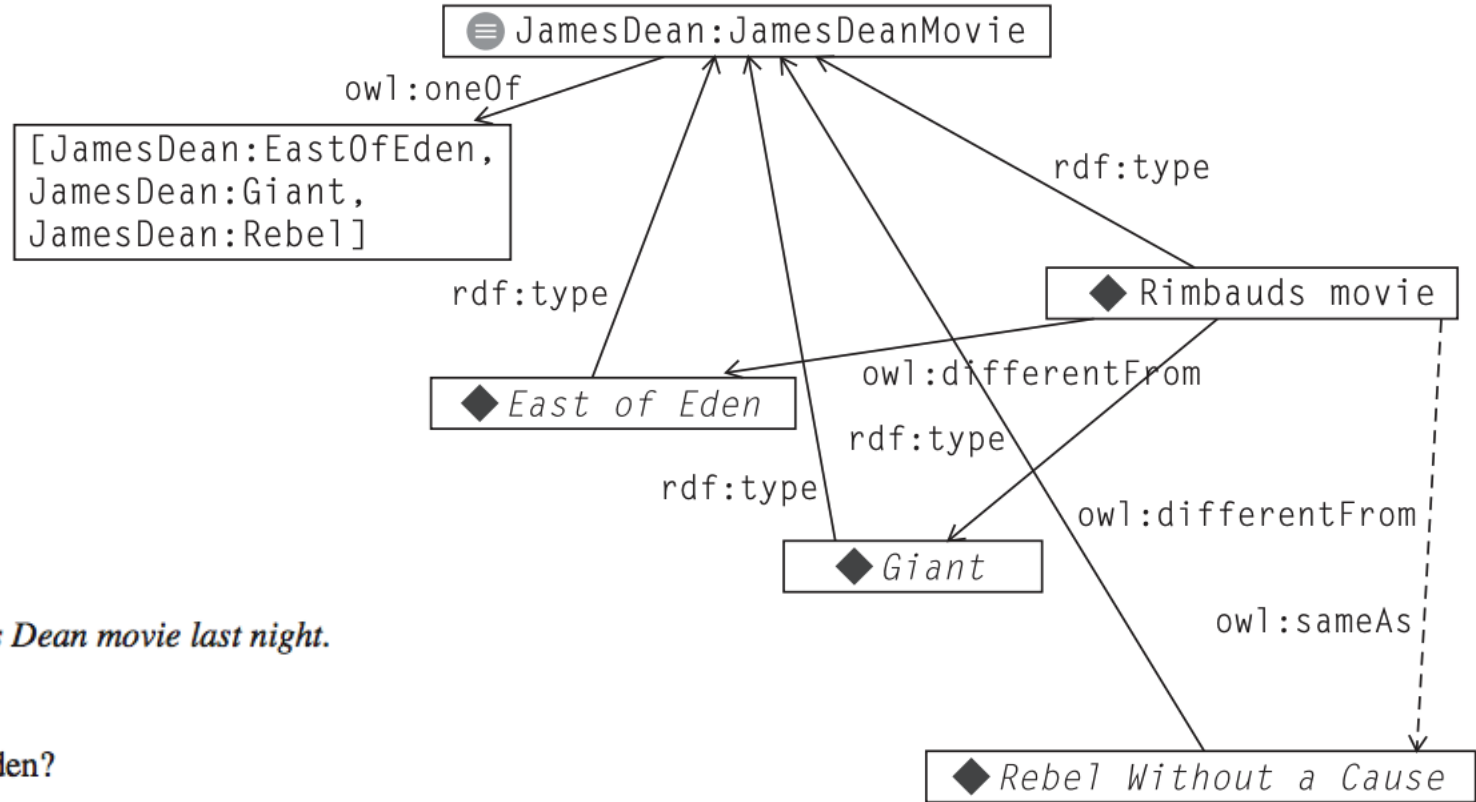
- In OWL 1, a class can be defined as the union of two other classes using owl:unionOf
- OWL 2 allows the declaration of a class representing a **disjoint union of a set of other classes**

```
<owl:Class ref:about="instruments">
  <owl:disjointUnionOf rdf:parseType="Collection">
    <owl:Class rdf:about="stringInstruments"/>
    <owl:Class rdf:about="percussionInstruments"/>
    <owl:Class rdf:about="pluckedInstruments"/>
    <owl:Class rdf:about="keyboardInstruments"/>
  </owl:disjointUnionOf>
</owl:Class>
```

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@base <http://example.org/>

<instruments> a owl:Class ;
  owl:disjointUnionOf (
    <stringInstruments>
    <percussionInstruments>
    <pluckedInstruments>
    <keyboardInstruments>
  ) .
```

Modeling Example: Class declaration as enumeration of individuals



RIMBAUD: I saw a James Dean movie last night.

ROCKY: Was it Giant?

RIMBAUD: No.

ROCKY: Was it East of Eden?

RIMBAUD: No.

ROCKY: James Dean only made three movies; it must have been Rebel Without a Cause.

RIMBAUD: Yes, it was.

- OWL 2 allows the definition of **disjoint properties**
 - two properties are disjoint if there are no two individuals that are interlinked by both properties.

```
<owl:ObjectProperty ref:about="hasParent">  
  <owl:propertyDisjointWith rdf:resource="hasChild" />  
</owl:ObjectProperty>
```

x1 hasParent x2 -> x1 hasChild x2 cannot exist

- Multiple mutually disjoint properties

```
<owl:AllDisjointProperties>  
  <owl:members rdf:parseType="Collection">  
    <owl:ObjectProperty ref:about="hasParent"/>  
    <owl:propertyDisjointWith rdf:resource="hasChild"/>  
    <owl:propertyDisjointWith rdf:resource="hasGrandchild"/>  
  </owl:members>  
</owl:AllDisjointProperties>
```

Allows making an assertion that no relationship exists, for a particular property, between an individual and another individual or value (therefore applicable both to object and data properties).

```
<rdf:Description>
  <rdf:type rdf:resource="owl:NegativePropertyAssertion"/>
  <owl:sourceIndividual rdf:resource="Marta"/>
  <owl:assertionProperty rdf:resource="hasNativeLanguage"/>
  <owl:targetIndividual rdf:resource="GermanLanguage"/>
</rdf:Description>
```

```
<rdf:Description>
<rdf:type rdf:resource="owl:NegativePropertyAssertion"/>
  <owl:sourceIndividual rdf:resource="Marta"/>
  <owl:assertionProperty rdf:resource="hasNumberOfHands"/>
  <owl:targetValue rdf:datatype="xsls:int">3</owl:targetValue>
</rdf:Description>
```

Allows making an assertion that no relationship exists, for a particular property, between an individual and another individual or value (therefore applicable both to object and data properties).

Property assertions: Marta ⏏ ⏏ ⏏ ⏏

Object property assertions +

Data property assertions +

Negative object property assertions +

■ **hasNativeLanguage** **GermanLanguage** ? @ × ○

Negative data property assertions +

■ **hasNumberOfHands** **"3"^^xsd:int** ? @ × ○

- Individuals can be identified uniquely with keys
- Keys defined on objectProperties, dataProperties or a combination of properties.

Persons with the same SSN number are the same:

```
<owl:Class rdf:about="Person">
  <owl:hasKey rdf:parseType="Collection">
    <owl:ObjectProperty rdf:about="hasSSN">
  </owl:hasKey>
</owl:Class>
```

All the students with the same name and the same birth are the same students

```
<owl:Class rdf:about="TUStudents">
  <owl:hasKey rdf:parseType="Collection">
    <owl:DatatypeProperty rdf:about="hasName"/>
    <owl:DatatypeProperty rdf:about="hasDOfBirth"/>
  </owl:hasKey>
</owl:Class>
```

- OWL 2 supports most of the basic XML Schema datatypes that were supported in OWL 1
 - Exceptions: *xsd:time*, *xsd:date*, *xsd:gYear*, *xsd:gMonth*, *xsd:gDay*, *xsd:gMonthDay*, *xsd:gYearMonth*
- New datatypes in OWL 2
 - *owl:real*, *owl:rational*, *rdf:PlainLiteral*, *rdf:XMLLiteral*, *xsd:dateTimeStamp*
- Additional possibilities for the restriction of concrete properties
 - Numbers: *xsd:maxExclusive*, *xsd:minInclusive*
 - Strings: *xsd:minLength*, *xsd:maxLength*, *xsd:length*, *xsd:pattern*
 - Cardinality: *DataMinCardinality*, *DataMaxCardinality*, and *DataExactCardinality*

- Metadata can be provided also by OWL annotations
- An OWL annotation simply associates property-value pairs with parts of an ontology, or the entire ontology itself.

```
<owl:Class rdf:about="Person">
  <rdfs:label>Represents the set of all people.</rdfs:label>
</owl:Class>

<owl:Class rdf:about="Man">
  <rdfs:subClassOf rdf:resource="Person"/>
</owl:Class>

<owl:Axiom>
  <owl:subject rdf:resource="Man"/>
  <owl:predicate rdf:resource="&rdfs;subClassOf"/>
  <owl:object rdf:resource="Person"/>
  <rdfs:label>States that every man is a person.</rdfs:label>
</owl:Axiom>
```

- Ontologies – refresher
- Advanced OWL Features
 - OWL2 – selected new features
 - **Property restrictions**
 - Property chaining
- Ontology Design Patterns
 - Modeling “part of”
 - Modeling n-ary relations
 - Modeling roles

A restriction describes a class of individuals based on the relationships that members of the class participate in. In other words, a restriction is a kind of class, in the same way that a named class is a kind of class [1].

- The OWL specification requires that **restrictions cannot be named** and must be defined using anonymous resources. This is a reasonable condition because restrictions are relevant only to the context of the class in which they are defined and never need to be referred to.
- Main type of restrictions:
 - Value restrictions: `owl:allValuesFrom`; `owl:someValuesFrom`; `owl:hasValue`
 - Cardinality restrictions: `owl:minCardinality`; `owl:maxCardinality`; `owl:cardinality`
 - OWL2: value and cardinality restrictions: `owl:minQualifiedCardinality`; `owl:maxQualifiedCardinality`; `owl:qualifiedCardinality`

[1] M. DeBellis, A Practical Guide To Building OWL Ontologies Using Protege 5 and Plugins, Edition 3.2.

<https://www.michaeldbellis.com/post/new-protege-pizza-tutorial>

An existential restriction describes a class of individuals that have *at least one (some)* relationship along a specified property to an individual that is a member of a specified class.

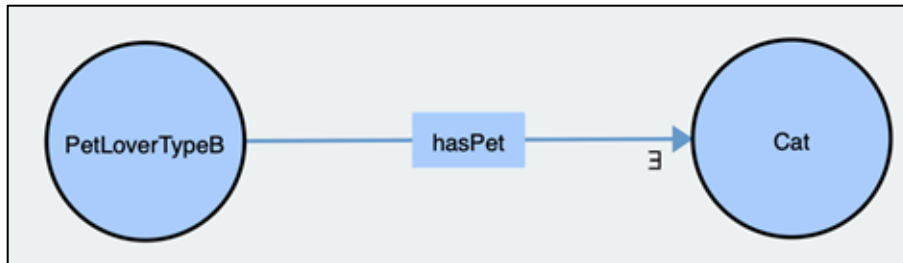
- To be a member of the class, the individual *must have at least one relation* along the given property to individuals that are members of a specific class (necessary condition).
- may be denoted by the existential quantifier ($\exists$). In OWL
`owl:someValuesFrom`.

```
<owl:Class rdf:about="Pizza">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="hasBase"/>
      <owl:someValuesFrom rdf:resource="PizzaBase"/>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

- Describes that there must exist an individual for property P and fixes its range to class C.

Meaning: **All** instances of PetLoverTypeB have **at least one** pet that is a cat.

... but the statement does not imply that an instance of PetLoverTypeB does not have other pets as well.



Valid Instances:

PetLoverTypeB has one cat.

PetLoverTypeB has 2 cats.

PetLoverTypeB has a cat, 2 dogs and a bird.

Invalid Instances:

PetLoverTypeB has a dog.

PetLoverTypeB has a bird and rabbit.

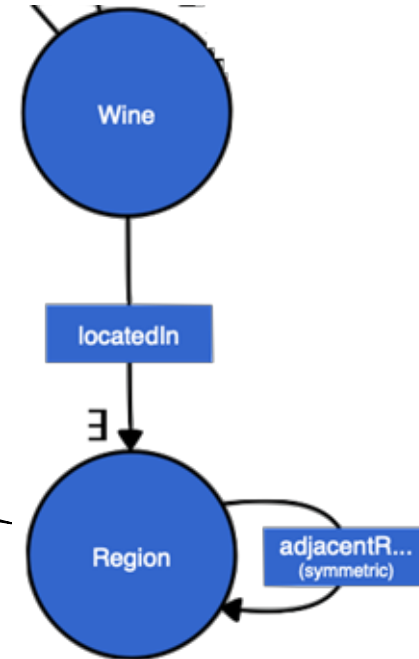
PetLoverTypeB has no pets.

Existential restriction (someValuesFrom):

- There **must be at least one** property with that value
- E.g., Wines are located in regions

There must be at least one **locatedIn** property whose value is **Region**.

[There must be at least one **locatedIn** property.]



Universal restriction over an object property expression: contains those individuals that are connected through an object property expression **only** to instances of a given class expression.

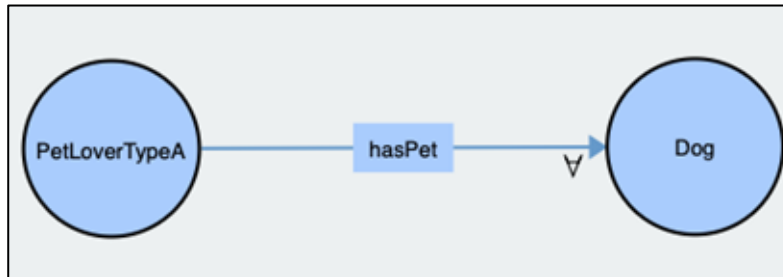
- for a given property, universal restrictions *do not specify the existence* of a relationship. They merely state that if a relationship exists for the property then it must be to individuals that are members of a specific class.
- may be denoted by the universal quantifier ($\forall$), which can be read as **only**. In OWL
`owl:allValuesFrom`

```
<owl:Class rdf:about="VegetarianPizza">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="hasTopping"/>
      <owl:allValuesFrom rdf:resource="VegetarianTopping"/>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

How about an individual that has no ***hasTopping*** relation? Will it be also considered a “Vegetarian” Pizza?

- Fixes all instances of a specific Class C as allowed range for property P.

Meaning: **If** an instance of PetLoverTypeA has a pet, it can **only** be a dog.
 ... but the statement does not imply that an instance of PetLoverTypeA must have a dog.



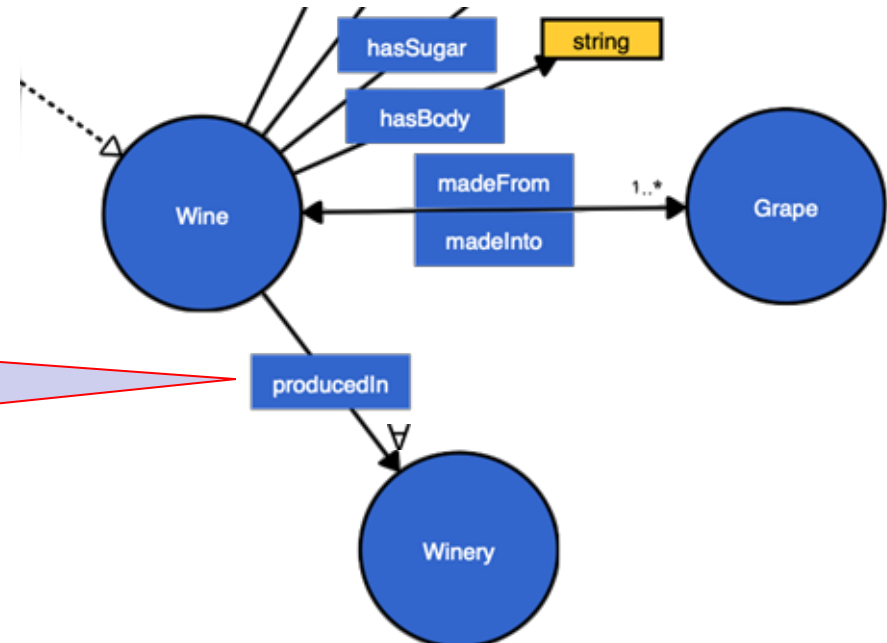
Valid Instances:

PetLoverTypeA has one dog.
 PetLoverTypeA 5 dogs.
 PetLoverTypeA has no pets.

Invalid Instances:

PetLoverTypeA has a cat.
 PetLoverTypeA has a dog and a cat.

- Universal restriction (allValuesFrom):
 - If a property is declared for an instance, all its values must be of a certain type
 - **Wines can only be produced in Wineries**



Wherever there is a **producedIn** property, all its values must be instances of **Winery**.
[There may be zero **producedIn** properties.]

A common pattern is combining universal and existential restrictions to define classes that have *at least one* relation of a property and all values of the property range over individuals of a certain class.

```
<owl:Class rdf:about="VegetarianPizza">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="hasTopping"/>
      <owl:someValuesFrom rdf:resource="VegetarianTopping"/>
    </owl:Restriction>
  </rdfs:subClassOf>
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="hasTopping"/>
      <owl:allValuesFrom rdf:resource="VegetarianTopping"/>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

OWL2 introduces a construct for constraining both the **number** and **type** of values a property should take for instances of a class → Qualified Cardinality Restriction

Qualified Cardinality Restrictions (Combining Existential And Universal Restrictions)

Qualified Cardinality Restrictions (QCR) is an alternative to expressing a combination of existential and universal restrictions as they allow constraining both the number and value types of a property for individuals of a class.

```
<owl:Class rdf:about="VegetarianPizza">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty ref:resource="hasTopping" />
      <owl:minQualifiedCardinality rdf:datatype="xsd:nonNegativeInteger">
        1
      </owl:minQualifiedCardinality>
      <owl:onClass rdf:resource="VegetarianTopping" />
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

QCR are more specific than cardinality restrictions in that they state not only the exact number of relations of type P that an individual must participate in, but also the class of objects within the restriction.

- In OWL:
 - owl:minQualifiedCardinality;
 - owl:maxQualifiedCardinality;
 - owl:qualifiedCardinality
- Example: an exam has at most 2 examiners which are professors

```
<owl:Class rdf:about="Exam">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty ref:resource="hasExaminer" />
      <owl:maxQualifiedCardinality rdf:datatype="xsd:nonNegativeInteger">
        2
      </owl:maxQualifiedCardinality>
      <owl:onClass rdf:resource="Professor" />
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

A **hasValue** restriction, describes an anonymous class of individuals that are related to another *specific individual* along a specified property

- In OWL: `owl:hasValue`

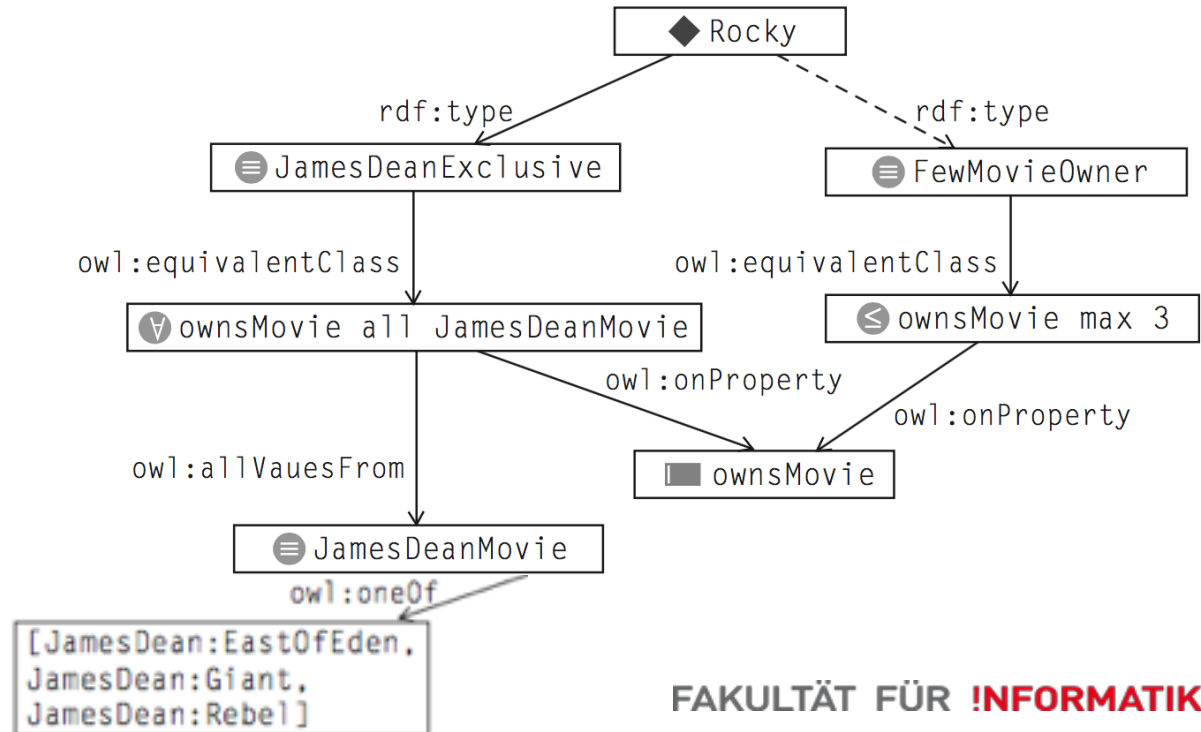
```
<owl:Class rdf:about="ItalianMozarella">
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="hasCountryOfOrigin"/>
      <owl:hasValue rdf:resource="Italy"/>
    </owl:Restriction>
  </rdfs:subClassOf>
</owl:Class>
```

Modeling Example: property restrictions

RIMBAUD: Do you own any James Dean movies?

ROCKY: They are the only ones I own.

RIMBAUD: Then I guess you don't own very many movies! No more than three.



- Ontologies – refresher
- Advanced OWL Features
 - OWL2 – selected new features
 - Property restrictions
 - **Property chaining**
- Ontology Design Patterns
 - Modeling “part of”
 - Modeling n-ary relations
 - Modeling roles

Allows to infer the existence of a property from a chain of properties. Enables property propagations among each others.

E.g., in life sciences one often needs to express that a characteristic of a part also affect the whole: *a fracture of the neck of the femur is a fracture of the femur.*

Example:

- **hasGrandparent** connects all individuals that are linked by a chain of exactly 2 **hasParent** properties
- $\text{hasParent} \circ \text{hasParent} \sqsubseteq \text{hasGrandparent}$
- Or as a rule: $\text{hasParent}(x, y) \wedge \text{hasParent}(y, z) \rightarrow \text{hasGrandparent}(x, z)$

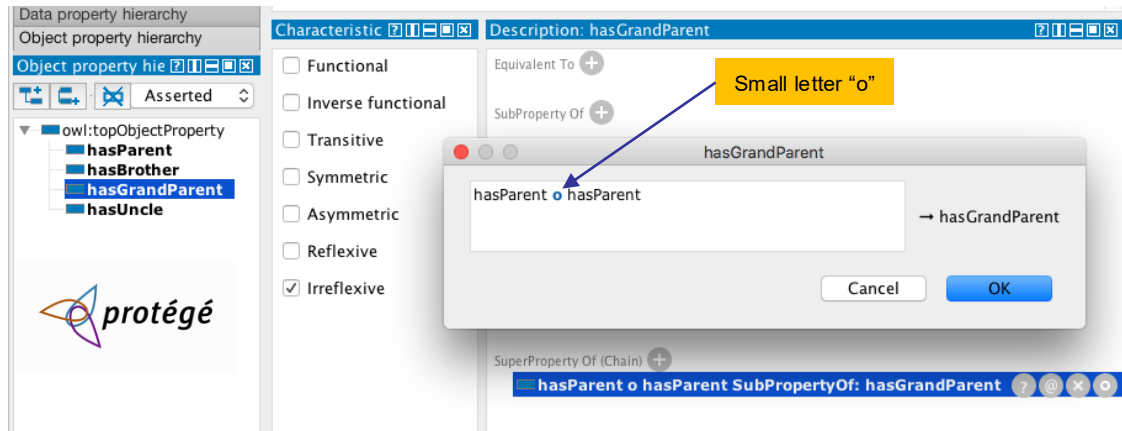
```
<rdf:Description rdf:about="hasGrandparent">
  <owl:propertyChainAxiom rdf:parseType="Collection">
    <owl:ObjectProperty rdf:about="hasParent"/>
    <owl:ObjectProperty rdf:about="hasParent"/>
  </owl:propertyChainAxiom>
</rdf:Description>
```

Allows to infer the existence of a property from a chain of properties. Enables property propagations among each others.

E.g., in life sciences one often needs to express that a characteristic of a part also affect the whole: *a fracture of the neck of the femur is a fracture of the femur.*

Example:

- **hasGrandparent** connects all individuals that are linked by a chain of exactly 2 **hasParent** properties
- $\text{hasParent} \circ \text{hasParent} \sqsubseteq \text{hasGrandparent}$
- Or as a rule: $\text{hasParent}(x, y) \wedge \text{hasParent}(y, z) \rightarrow \text{hasGrandparent}(x, z)$



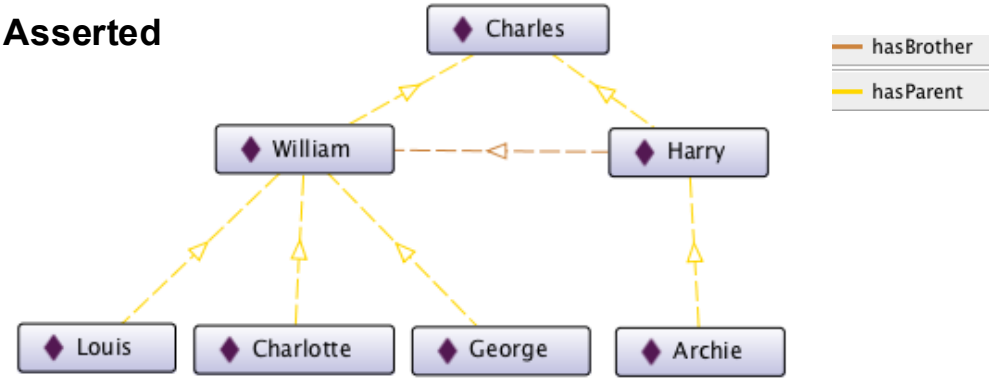
OWL 2 – Property Chaining

Example 1 (*): modeling of genealogical relations.



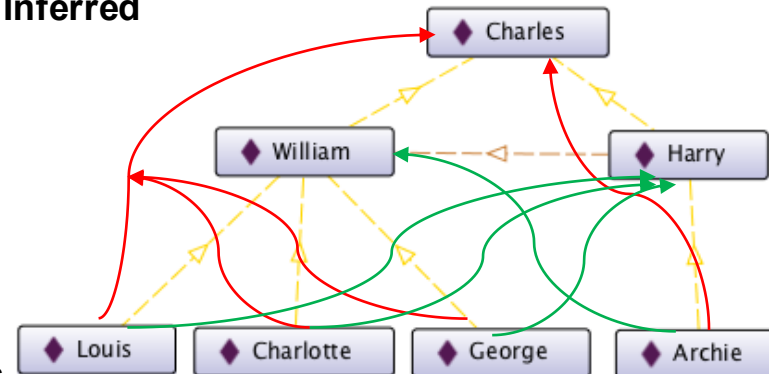
Credits: Tatiana Ayazo/rd.com, Shutterstock

Asserted



- $\text{hasParent} \circ \text{hasParent} \sqsubseteq \text{hasGrandparent}$
- $\text{hasParent} \circ \text{hasBrother} \sqsubseteq \text{hasUncle}$

Inferred

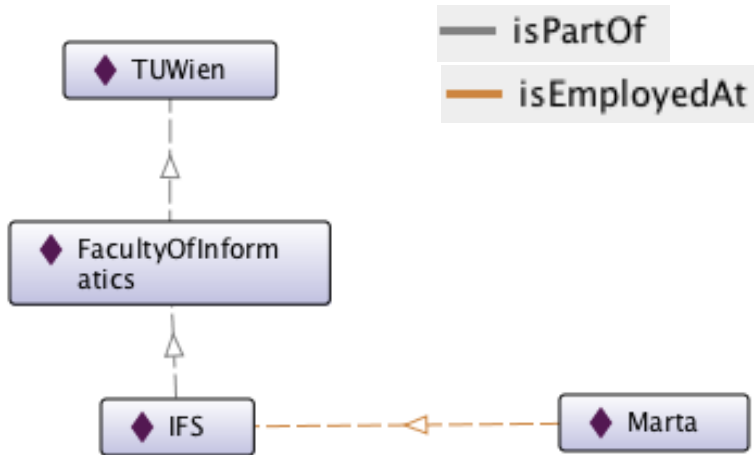


(*) might be outdated, only for demo purposes

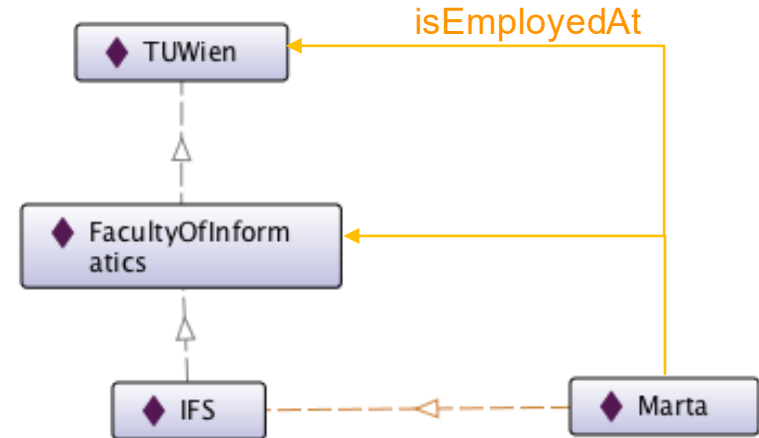
Example 2: modeling relations that “propagate” across part-of hierarchies.

E.g., a person employed in a department is also employed by the organization to which the department belongs.

Asserted



Inferred

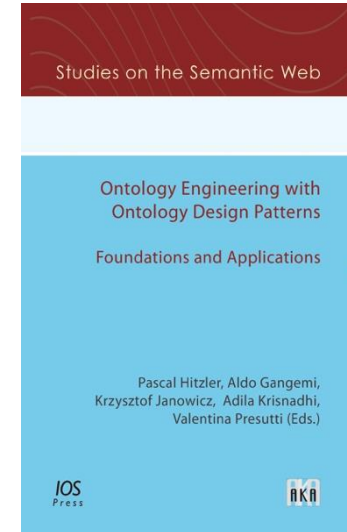
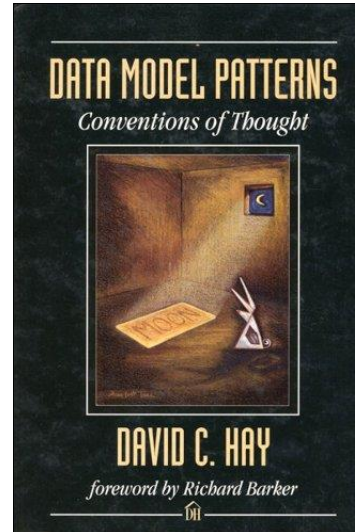
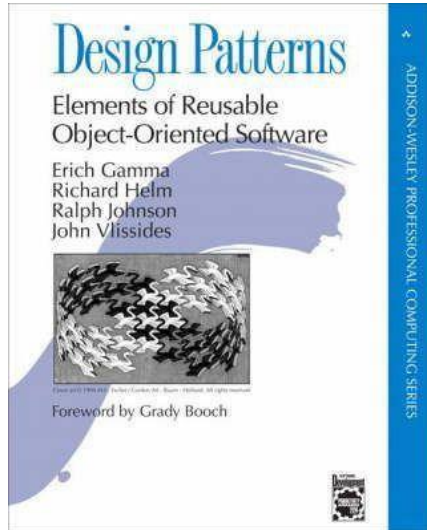


- $isEmployedAt \circ isPartOf \sqsubseteq isEmployedAt$
- $isEmployedAt(x, y) \wedge isPartOf(y, z) \rightarrow isEmployedAt(x, z)$

- Ontologies – refresher
- Advanced OWL Features
 - OWL2 – selected new features
 - Property restrictions
 - Property chaining
- **Ontology Design Patterns**
 - Modeling “part of”
 - Modeling n-ary relations
 - Modeling roles

*An **ontology design pattern (ODP)** is a reusable successful solution to a recurrent modeling problem.*

Patterns are used in many areas as "templates" or abstract descriptions encoding best practices of some field





The screenshot shows the homepage of the Ontology Design Patterns .org (ODP) website. The browser address bar displays the URL ontologydesignpatterns.org/wiki/Ontology_Design_Patterns. The page features a navigation sidebar on the left with sections for navigation, contribute, and help. The main content area includes a header with the site name, a description of the portal, and a 'What's new' section listing recent workshops and tutorials. At the bottom, there are three columns: Navigation, Contribute, and News, each with a brief description and a link to the relevant content.

Navigation

- Main page
- ODPA
- List patterns
- Pattern types
- Modeling Issues
- Domains
- Training
- Events

Contribute

- Submit a pattern
- Submit an exemplary ontology
- Post a modeling issue
- Review a pattern
- Feedback about the portal
- Request an ODP account

Help

- About ODP
- What is a pattern?
- What is an exemplary ontology?

Ontology Design Patterns .org (ODP)

OntologyDesignPatterns.org is a **Semantic Web** portal dedicated to ontology design patterns (ODPs), run by the [ODPA](#).

The portal was started under the [NeOn](#) project.

What's new

- The 10th Workshop on Ontology Design and Patterns ([WOP2019](#)) will be held at [ISWC2019](#) (October 26 or 27) in Auckland, New Zealand.
- The 9th Workshop on Ontology Design and Patterns ([WOP2018](#)) was held at [ISWC2018](#) (October 9) in Monterey, CA, USA.
- The tutorial [Methods and Tools for Modular Ontology Modeling](#) was held at [ISWC2018](#) (October 8) in Monterey, CA, USA.
- The 8th Workshop on Ontology Design and Patterns ([WOP2017](#)) was held at [ISWC2017](#) (October 21) in Vienna, Austria.
- The tutorial [Modular Ontology Modeling with Ontology Design Patterns](#) is held at [ESWC 2017](#) (May 28) in Portoroz, Slovenia.
- The 7th Workshop on Ontology and Semantic Web Patterns ([WOP2016](#)) will be held at [ISWC2016](#) (October 18) in Kobe, Japan.
- The tutorial [Ontology Design Patterns for Linked Data Publishing](#) is being held at [ISWC2016](#) (October 17) in Kobe, Japan.
- The Ontology Design and Patterns Association was formed - for information see [the ODPA page](#).

Navigation

List of Patterns

You can find lists here, detailing all available ontology design patterns.

Contribute

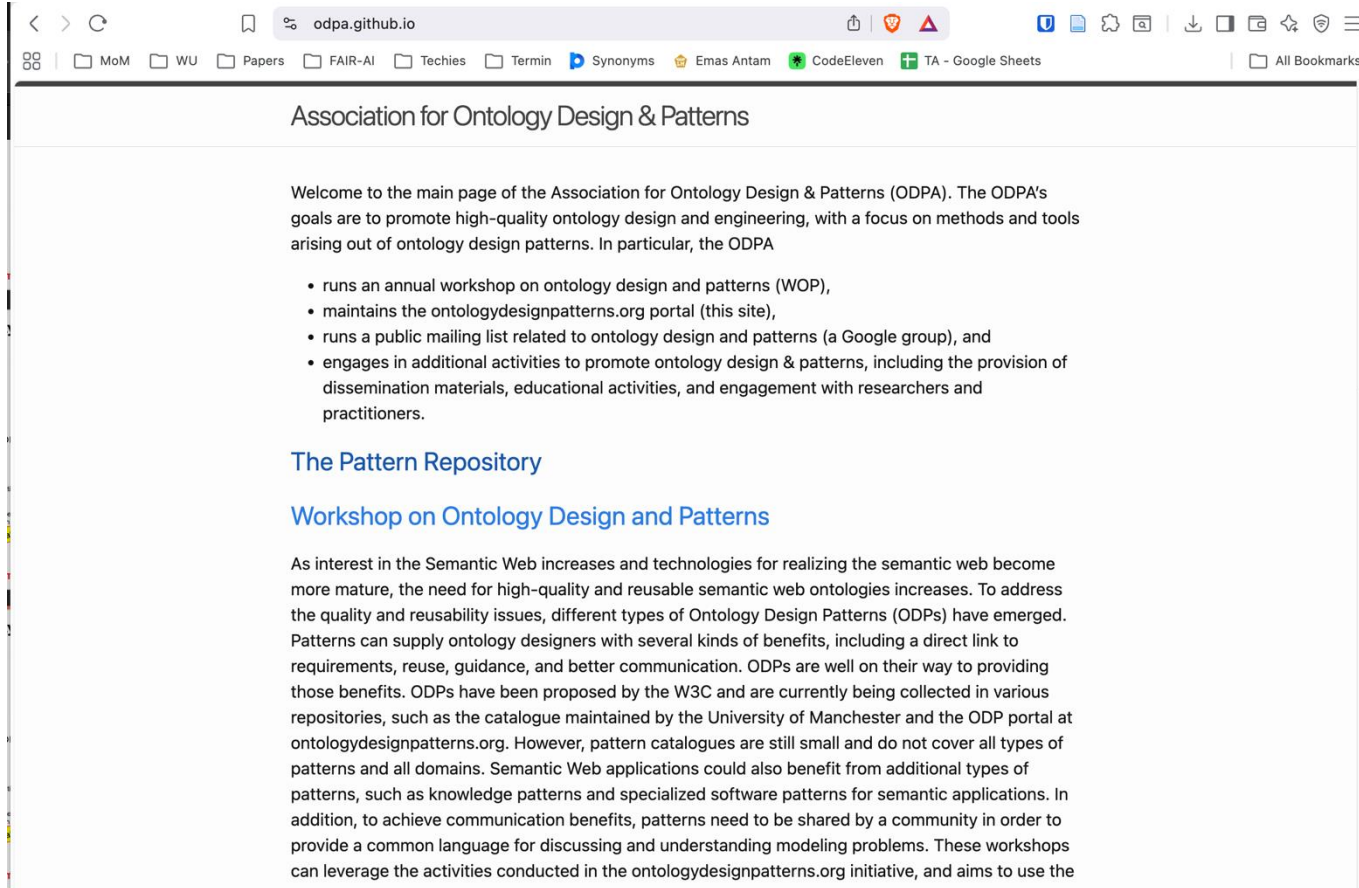
Submit Pattern

Start here if you want to submit an ontology pattern.

News

Latest ODP News!

WOP 2019 to be held at ISWC 2019 in New Zealand



The screenshot shows a web browser displaying the homepage of the Association for Ontology Design & Patterns (ODPA). The browser's address bar shows 'odpa.github.io'. The page title is 'Association for Ontology Design & Patterns'. The main content area includes a welcome message, a list of activities, and links to the pattern repository and workshop.

Association for Ontology Design & Patterns

Welcome to the main page of the Association for Ontology Design & Patterns (ODPA). The ODPA's goals are to promote high-quality ontology design and engineering, with a focus on methods and tools arising out of ontology design patterns. In particular, the ODPA

- runs an annual workshop on ontology design and patterns (WOP),
- maintains the ontologydesignpatterns.org portal (this site),
- runs a public mailing list related to ontology design and patterns (a Google group), and
- engages in additional activities to promote ontology design & patterns, including the provision of dissemination materials, educational activities, and engagement with researchers and practitioners.

[The Pattern Repository](#)

[Workshop on Ontology Design and Patterns](#)

As interest in the Semantic Web increases and technologies for realizing the semantic web become more mature, the need for high-quality and reusable semantic web ontologies increases. To address the quality and reusability issues, different types of Ontology Design Patterns (ODPs) have emerged. Patterns can supply ontology designers with several kinds of benefits, including a direct link to requirements, reuse, guidance, and better communication. ODPs are well on their way to providing those benefits. ODPs have been proposed by the W3C and are currently being collected in various repositories, such as the catalogue maintained by the University of Manchester and the ODP portal at ontologydesignpatterns.org. However, pattern catalogues are still small and do not cover all types of patterns and all domains. Semantic Web applications could also benefit from additional types of patterns, such as knowledge patterns and specialized software patterns for semantic applications. In addition, to achieve communication benefits, patterns need to be shared by a community in order to provide a common language for discussing and understanding modeling problems. These workshops can leverage the activities conducted in the ontologydesignpatterns.org initiative, and aims to use the

In this lecture:

- Part-of ODPs
- N-ary relation ODPs
- Role ODPs

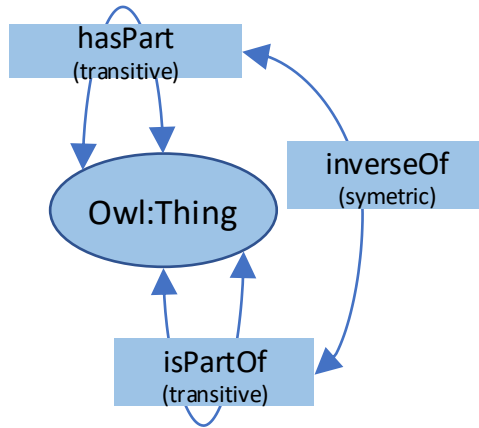
Gangemi, Aldo, and Valentina Presutti. "Ontology design patterns." In *Handbook on ontologies*, pp. 221-243. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.

ReadingMaterial

- Ontologies – refresher
- Advanced OWL Features
 - OWL2 – selected new features
 - Property restrictions
 - Property chaining
- Ontology Design Patterns
 - **Modeling “part of”**
 - Modeling n-ary relations
 - Modeling roles

- **part-whole** relations are important in several domains, e.g., engineering and life sciences
- Typical applications:
 - inventory applications: given an engineering object, report all its parts and the subparts of those;
 - Other applications where the composition of an engineering object is explored one level at a time (i.e., only the **direct** components of the object are shown at any time).
- Yet, RDF(S) and OWL, do not provide built-in primitives for modelling part-whole relations as they do for modelling subsumption relations (e.g., `rdfs:subClassOf`).

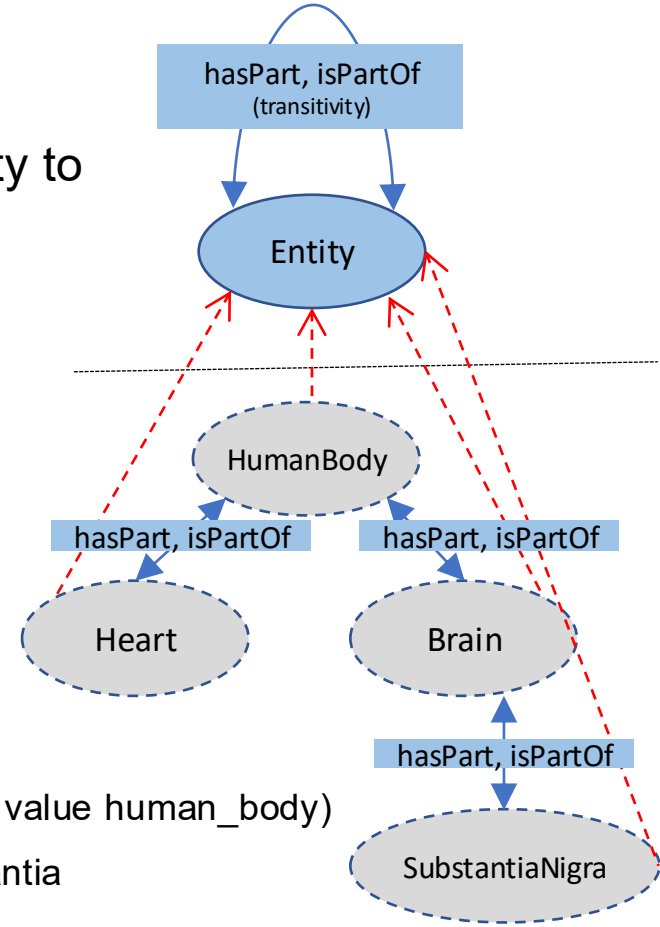
- Transitive modeling of part-whole relation
- Transitively returns all sub-parts (no possibility to return only a single level of sub-parts)



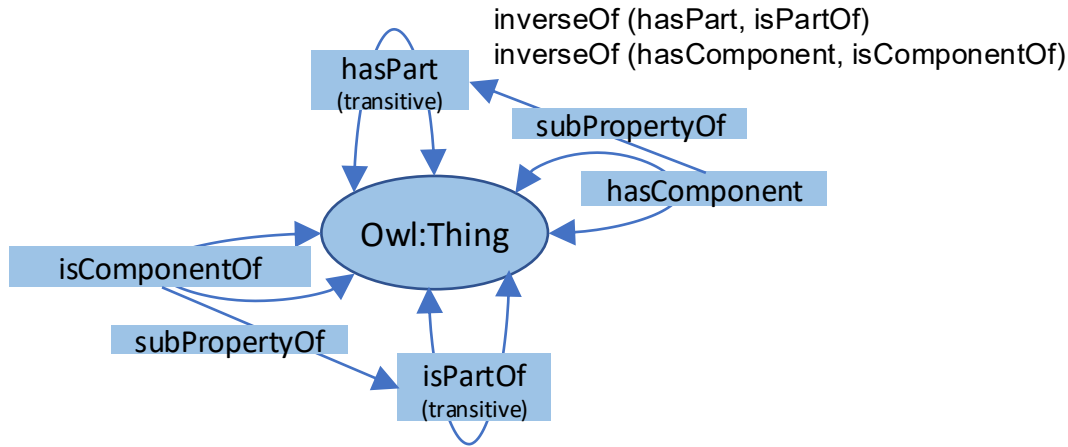
DL query: Entity and ('is part of' value human_body)

Response: Heart, Brain, Substantia

DEMO



- allows to distinguish between parts and proper parts
- relation of proper part is *not transitive* (*hasComponent*), and implies a simple *hasPart* relation, which is *transitive*.



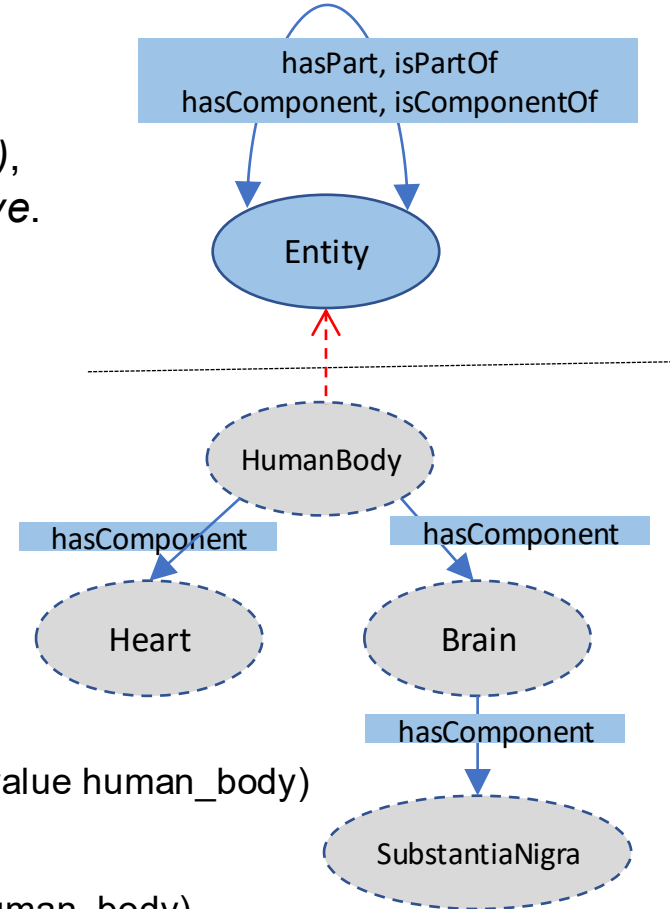
DL query: Entity and (isComponentOf value human_body)

Response: Heart, Brain

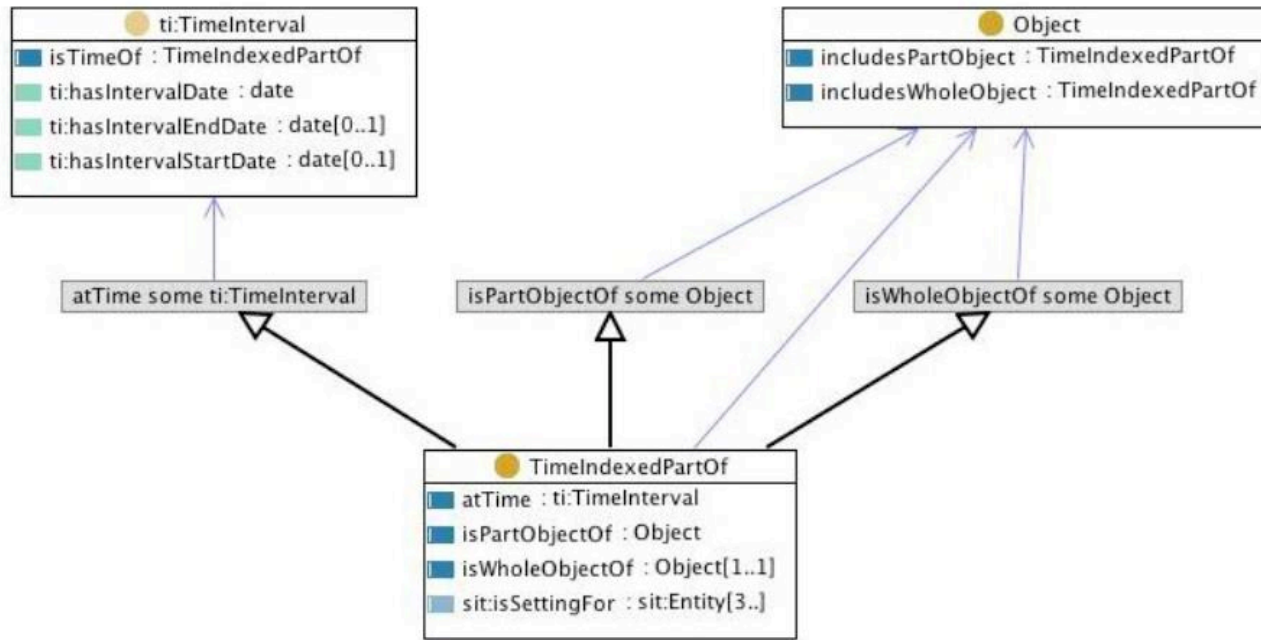
DL query: Entity and (isPartOf value Human_body)

Response: Heart, Brain, SubstantiaNigra

DEMO



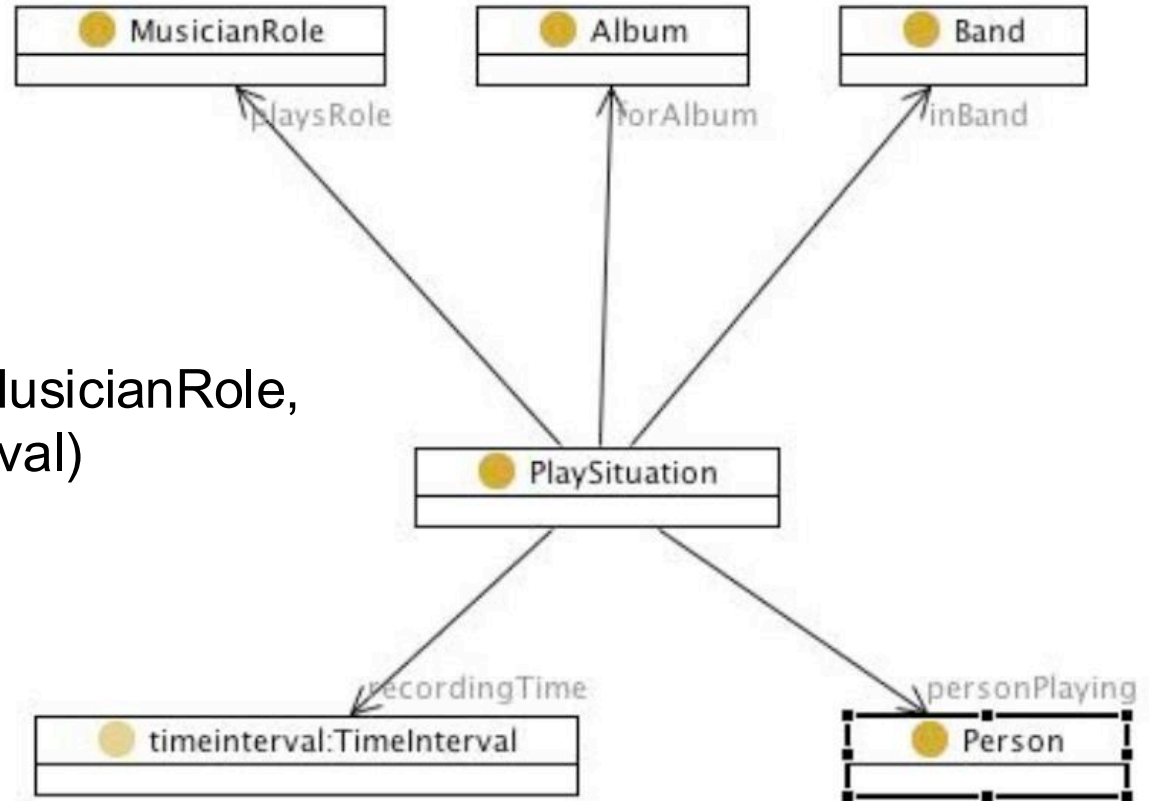
- Allows modelling of a situation in which an object is part of another for a specified time interval
- An n -ary relationships to establish time-based part-whole relations between a part, a whole, and a time interval

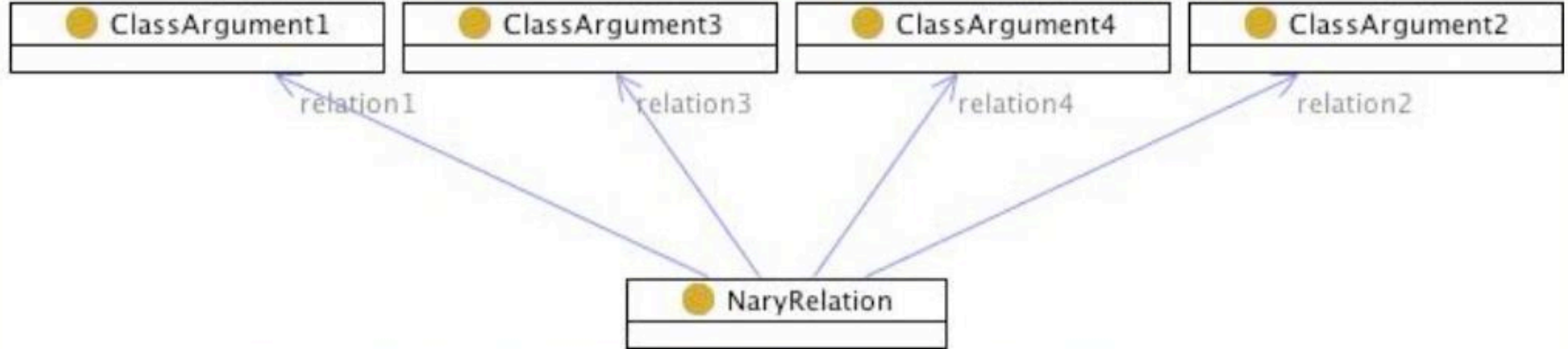


- Ontologies – refresher
- Advanced OWL Features
 - OWL2 – selected new features
 - Property restrictions
 - Property chaining
- Ontology Design Patterns
 - Modeling “part of”
 - **Modeling n-ary relations**
 - Modeling roles

- N-ary relations relating several entities are common in practice but challenging to model in the triple-based paradigm of semantic web languages
- Example:
 - Chad Smith was the drum player of Red Hot Chili Peppers when they recorded their album Stadium Arcadium from September 2004 to December 2005.
 - **OR**: A person plays a certain role in a band during an album recording, taking place during a certain time interval
 - **OR**: PlaySituation(Person, MusicianRole, Band, Album, TimeInterval)

PlaySituation(Person, MusicianRole, Band, Album, TimeInterval)

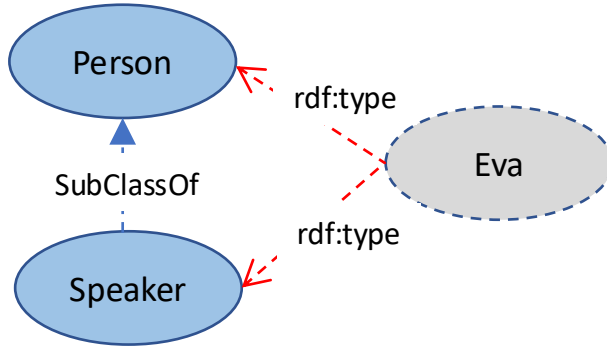




- Ontologies – refresher
- Advanced OWL Features
 - OWL2 – selected new features
 - Property restrictions
 - Property chaining
- Ontology Design Patterns
 - Modeling “part of”
 - Modeling n-ary relations
 - **Modeling roles**

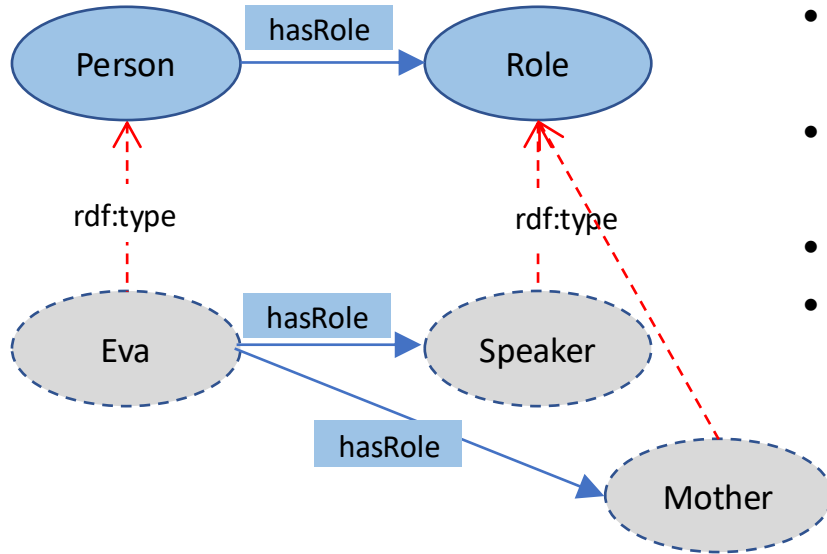
- Objects can play different roles in different situation
 - Eva will be a speaker at the WOP workshop.
 - She greeted us all in her various roles of mother, friend, and daughter.
 - Chad Smith was the drum player of Red Hot Chili Peppers
- There are various modeling approaches with their own pros and cons

(1) Roles as Classes



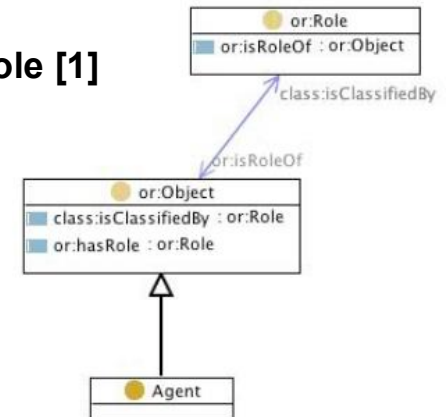
- Roles are described at TBox level
- Class taxonomy is bigger - a class for each role
- Class taxonomy is entangled - multi-typing
- *ABox is smaller – same individual, several (role) types*
- Roles cannot be indexed in terms of space and time
- Facts about roles cannot be expressed e.g. “Roles in UniBo can be student, professor, researcher”

(2) Roles as Individuals



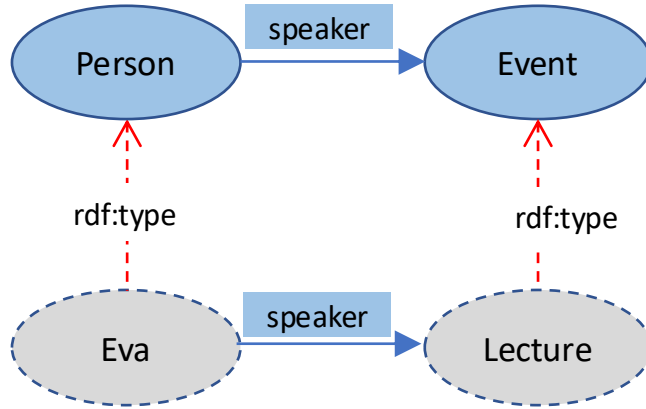
- More expressive than (1)
- Roles are described at ABox level
- *Class taxonomy is smaller, Abox is bigger*
- Facts on roles can be asserted
- Roles can be indexed in terms of time and space - through n-ary relations

ODP: AgentRole [1]

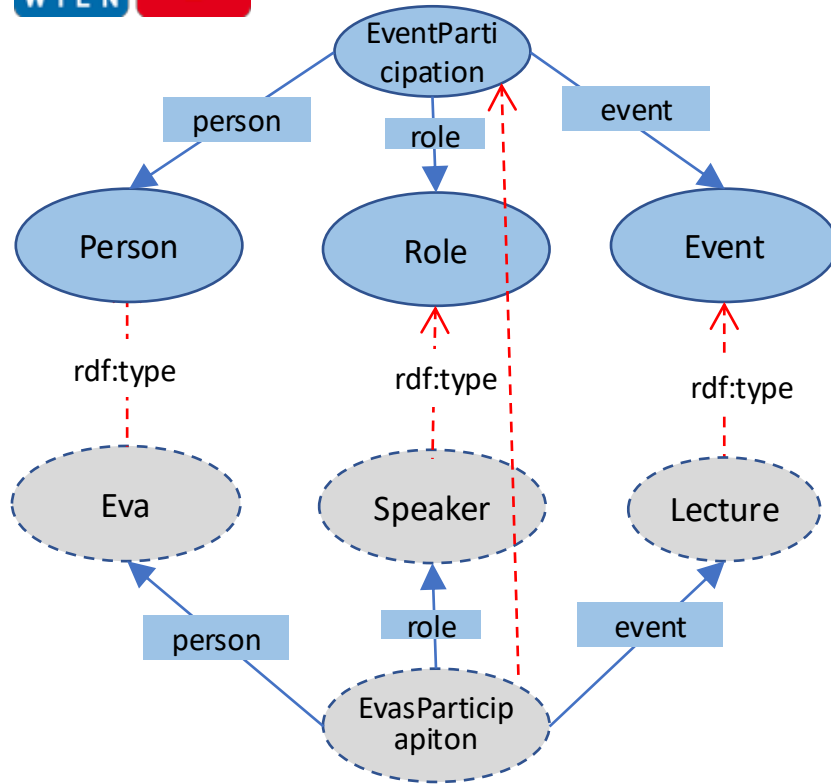


[1] <https://odpa.github.io/patterns-repository/AgentRole/AgentRole>

(3) Roles as Properties



- The semantics of “having a role” is embedded in the name of a property
- Objects are not explicitly related to their roles, they are related to other things through a property expressing an action they perform, a role they play
- Common pattern for modeling roles on web of data
- *Smaller taxonomy of classes*
- *Bigger taxonomy of properties*
- Roles cannot be indexed in terms of space and time
- Semantics of roles is implicit (embedded in a property name)
- Facts about roles cannot be expressed



(4) Roles as N-ary relations

- Participation in roles modeled as N-ary relation
- *Expressive, but larger taxonomy of classes, bigger taxonomy of properties*
- Roles can be indexed in terms of space and time
- Facts about roles cannot be expressed

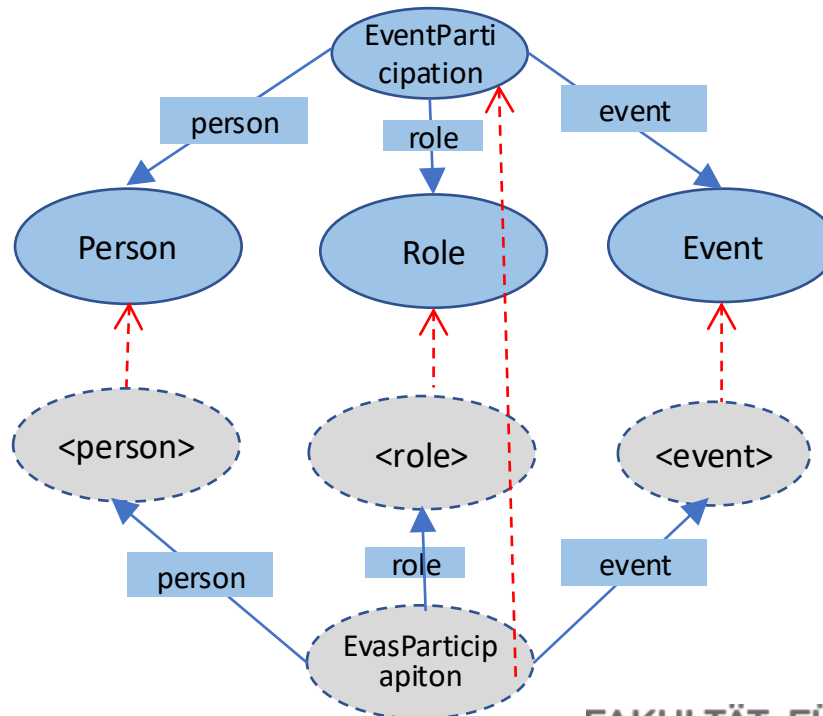
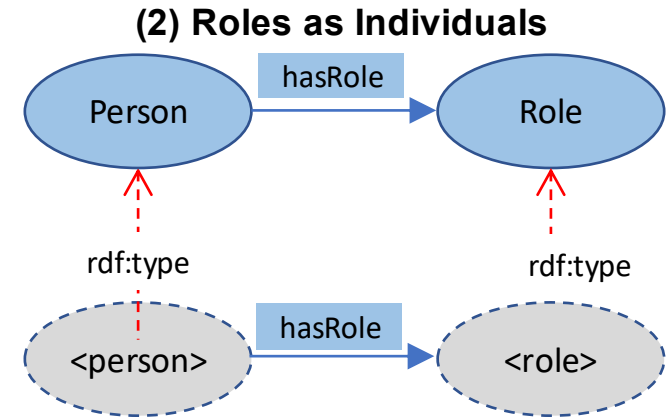
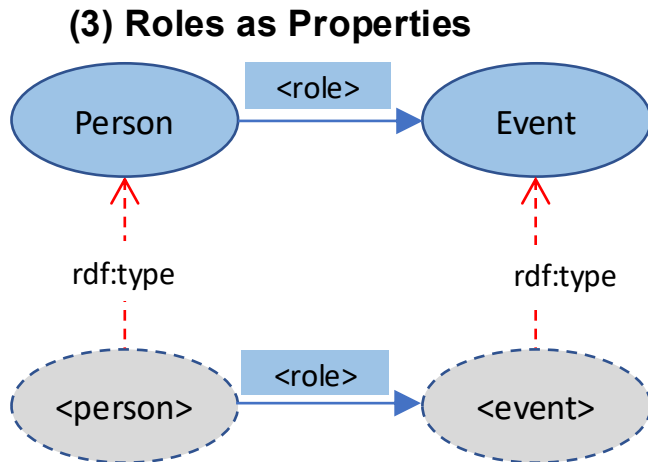
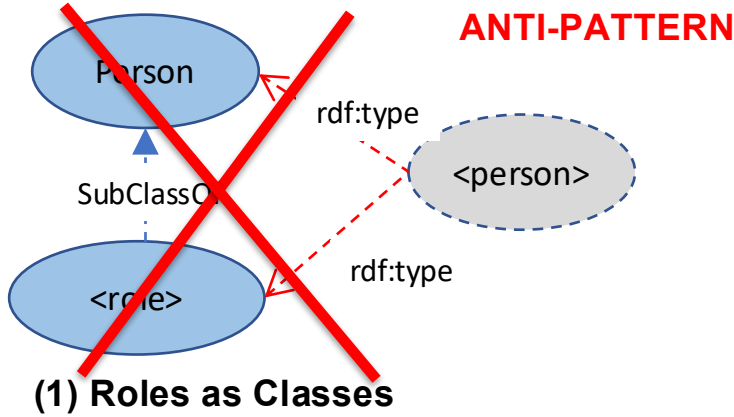
Related ODPs:

- [1] Time-indexed person role-pattern
- [2] Time-place-indexed object-role

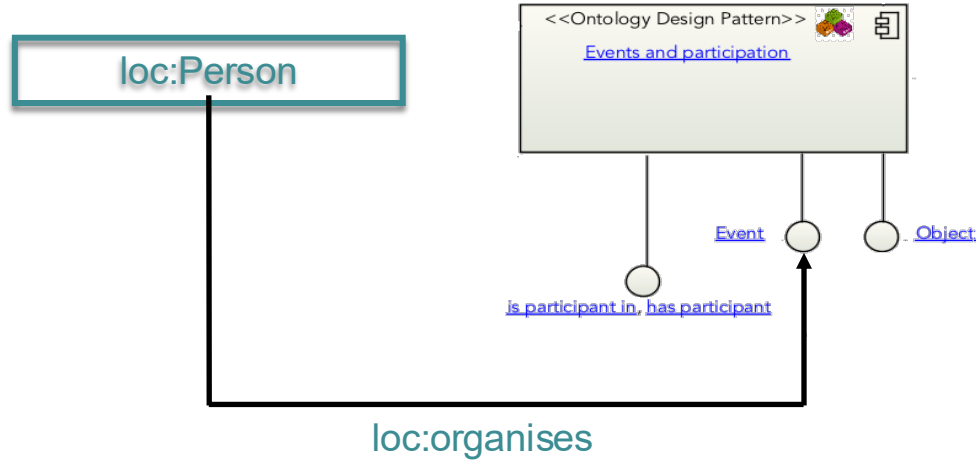
[1] https://odpa.github.io/patterns-repository/Time_indexed_person_role/Time_indexed_person_role

[2] <http://www.ontologydesignpatterns.org/cp/owl/dul/timeplaceindexedobjectrole.owl>

Summary – Roles

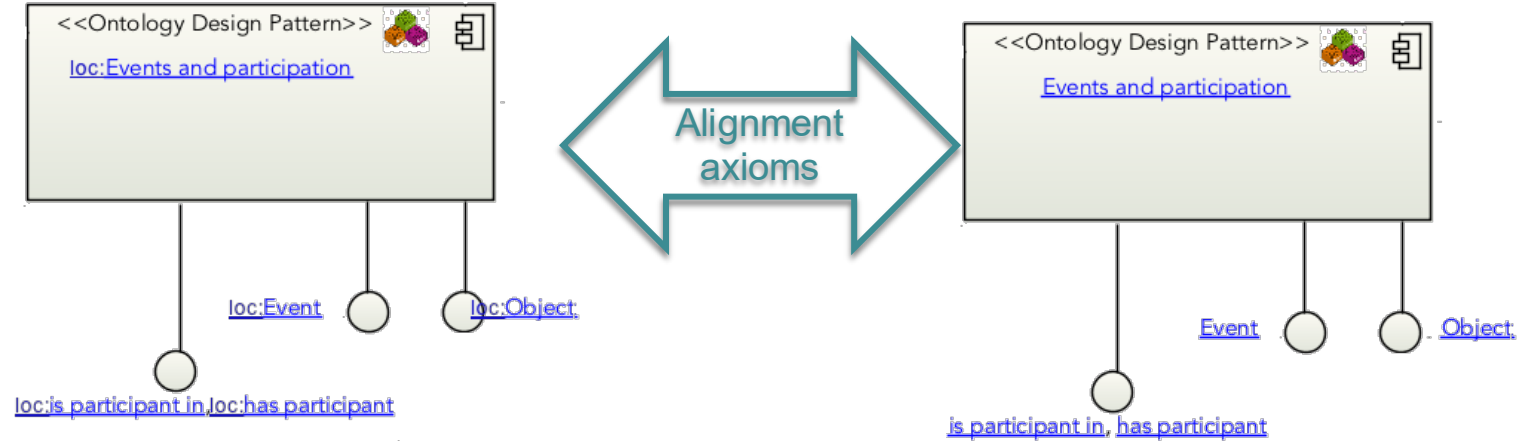


Reusing ODPs: direct reuse



Advantages	Disadvantages
- Stability and sustainability - Modularity - Interoperability - ODP are unlikely to change - Easy to re-design in case of changes	- Dependency on external module - Mitigated risk of instability

Reusing ODPs: indirect reuse

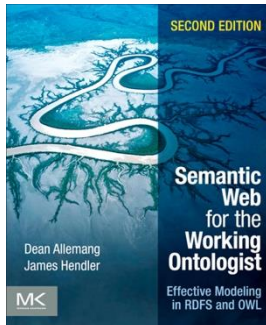


Advantages	Disadvantages
- Dependency on external changes is limited to alignment axioms - Easier re-design for fixing issues due to external changes - Stability and sustainability - Modularity and interoperability	Effort for replicating the ODP implementation

- M. Horridge et al., A Practical Guide To Building OWL Ontologies Using Protege 4 and CO-ODE Tools, Edition 1.3. https://www.academia.edu/download/56546069/ProtegeOWLTutorialP4_v1_3.pdf
- OWL2: <http://www.w3.org/TR/owl2-primer/>
- “OWL 2: The Next Step for OWL - Web Semantics”, Bernardo Cuenca Grau, Ian Horrocks, Boris Motik, Bijan Parsia, Peter Patel-Schneider, Ulrike Sattler.
- Gangemi, Aldo, and Valentina Presutti. "Ontology design patterns." In *Handbook on ontologies*, pp. 221-243. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.



- **Ontology Design Patterns . org (ODP)** <http://ontologydesignpatterns.org>
- <https://github.com/odpa/patterns-repository> ; <https://odpa.github.io/>



- Chapter 12: Counting and sets in OWL

W3C Editor's Draft

TABLE OF CONTENTS

- Abstract**
- Status of This Document**
Set of Documents
- 1. Introduction**
 - 1.1 Graph-based Abstract Data Model
 - 1.2 Resources and Statements
 - 1.3 The Referent of an `IRI`
 - 1.4 RDF Vocabularies and Namespace IRIs
 - 1.5 Triple Terms and Reification
 - 1.6 RDF and Change over Time
 - 1.7 Working with Multiple RDF Graphs
 - 1.8 Equivalence, Entailment and Inconsistency
 - 1.9 RDF Documents and Syntaxes
 - 1.10 RDF Version Announcement
- 2. Conformance**
 - 2.1 Version Labels
 - 2.2 Strings in RDF
- 3. RDF Graphs**
 - 3.1 Triples
 - 3.2 RDF Terms
 - 3.3 IRIs
 - 3.4 Literals

RDF 1.2 Concepts and Abstract Data Model

W3C Editor's Draft 03 March 2026



▼ More details about this document

This version:
<https://w3c.github.io/rdf-concepts/spec/>

Latest published version:
<https://www.w3.org/TR/rdf12-concepts/>

Latest editor's draft:
<https://w3c.github.io/rdf-concepts/spec/>

History:
<https://www.w3.org/standards/history/rdf12-concepts/>
[Commit history](#)

Test suite:
<https://w3c.github.io/rdf-tests/rdf/rdf12/>

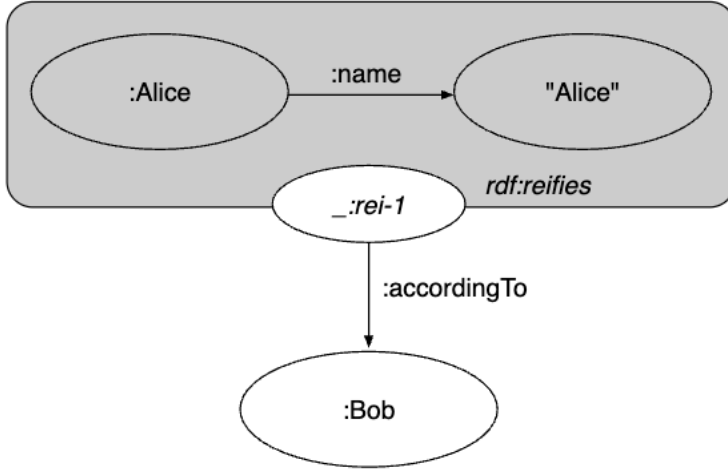
Implementation report:
<https://w3c.github.io/rdf-tests/rdf/rdf12/reports/>

Latest Recommendation:
<https://www.w3.org/TR/rdf11-concepts>

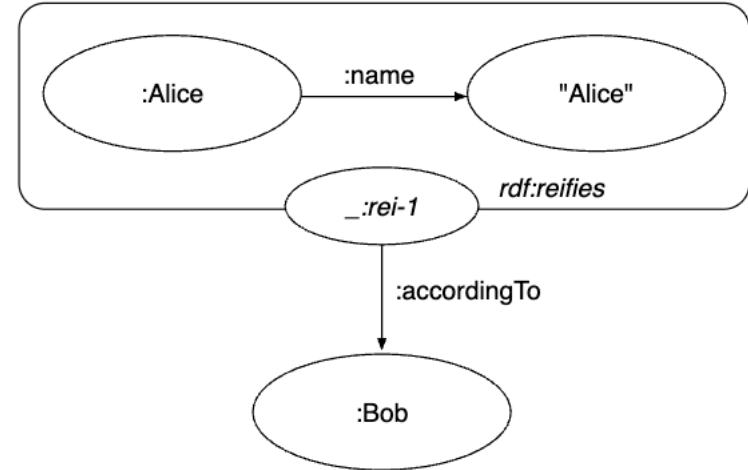
Editors:
Gregg Kellogg (until 2025-09-06), *in memoriam*
Olaf Hartig
Pierre-Antoine Champin
Andy Seaborne

<https://w3c.github.io/rdf-concepts/spec/>

Key feature: Reifications



An [RDF graph](#) containing a [triple](#) that references an unasserted [triple term](#) (with grey background) via a [reifier](#).



An [RDF graph](#) containing a [triple](#) that references an [triple term](#), which is also asserted, via a [reifier](#).

- Common problem:
 - `:A :p :B`
(but what if you need to say things about the relationship?)
- Standard reification:
 - `[] a rdf:Statement ; rdf:subject :A ;
rdf:predicate :p ; rdf:object :B`
- Typical solution – “lift” the relationship to be its own node:
 - `[ ] a :p_as_node ; :source :A ; :sink :B`
(for all intents and purposes equivalent with the standard reification)

- Triples in an RDF graph are unique
 - `:Elizabeth :married :Richard`
 - We want to qualify this with a year
 - but... Elizabeth married Richard twice (1964 and 1975)

- New type of term: **Triple Term**
 - (The other ones are IRI, blank node, and literal)
 - A triple term can be thought of as an un-asserted triple
- Triple terms can be given an identifier (a “reifier”) via a new predicate `rdf:reifies`
 - `:Marriage1 rdf:reifies <<( :Elizabeth :married :Richard )>>`
 - `:Marriage1` is now an “occurrence” of the original triple
 - Triple terms cannot be used in the subject position of a triple
- A triple term can have multiple reifiers → multiple “occurrences”
- A reifier can identify multiple triple terms

- The W3C working group have extended the RDF serialization formats to include triple terms
 - (except RDF/XML)
 - In extended syntaxes, triple terms look like this:
$$<< (\text{ :A } :p \text{ :B }) >>$$
 - In addition, some syntactic sugar is provided
- Without the use of triple terms, the syntaxes remain (mostly) the same as in RDF 1.1

```
<< :Elizabeth :married :Richard ~ :Marriage1 >>  
    :date "1964-03-15"^^xsd:date .
```

- Fully expanded, this is
 - :Marriage1 rdf:reifies
 <<(:Elizabeth :married :Richard)>> .
 - :Marriage1 :date "1964-03-15"^^xsd:date .

- Note that the reified triple is not asserted
 - (“not asserted” == “not part of the graph”)

Adapted from <https://www.lassila.org/publications/2025/lassila-gschwend-kgc2025.pdf>

```
:Elizabeth :married :Richard ~ :Marriage1 { | :date "1964-03-15"^^xsd:date | }
```

- This is syntactic sugar: the reified triple is also asserted, so this is the same as
 - `:Elizabeth :married :Richard .`
 - `<< :Elizabeth :married :Richard ~ :Marriage1 >>`
`:date "1964-03-15"^^xsd:date`
- which is the same as
 - `:Elizabeth :married :Richard .`
 - `:Marriage1 rdf:reifies <<( :Elizabeth :married :Richard )>> .`
 - `:Marriage1 :date "1964-03-15"^^xsd:date .`

One triple, two marriages

```
:Elizabeth :married :Richard .  
<< :Elizabeth :married :Richard ~ :Marriage1 >>  
    :date "1964-03-15"^^xsd:date .  
<< :Elizabeth :married :Richard ~ :Marriage2 >>  
    :date "1975-10-10"^^xsd:date .
```

You could also use the annotation syntax to achieve this

Summary and the Current Status of RDF 1.2

- Standard reification in RDF has been misunderstood and considered awkward
- RDF is criticized for the lack of “edge properties”
- RDF-star, now renamed RDF 1.2, simplifies the reification process
- Syntax and semantics have been extended for a new term: **triple term**
- Work is nearing completion

- Group Forming
- For next class:
 - Choose domain
 - Start extending/enriching your ontology
 - Make use of the reasoner to see how semantics are used.